

FRE6871 R in Finance

Lecture#4, Fall 2025

Jerzy Pawlowski jp3900@nyu.edu

NYU Tandon School of Engineering

November 17, 2025



NYU

**TANDON SCHOOL
OF ENGINEERING**

The Logistic Function

The *logistic* function expresses the probability of a numerical variable ranging over the whole interval of real numbers:

$$p(x) = \frac{1}{1 + \exp(-\lambda x)}$$

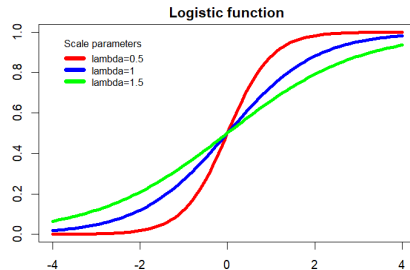
Where λ is the scale (dispersion) parameter.

The *logistic* function is often used as an activation function in neural networks, and logistic regression can be viewed as a perceptron (single neuron network).

The *logistic* function can be inverted to obtain the *Odds Ratio* (the ratio of probabilities for favorable to unfavorable outcomes):

$$\frac{p(x)}{1 - p(x)} = \exp(\lambda x)$$

The function `plogis()` gives the cumulative probability of the *Logistic* distribution,



```
> lambdav <- c(0.5, 1, 1.5)
> colorv <- c("red", "blue", "green")
> # Plot three curves in loop
> for (it in 1:3) {
+   curve(expr=plogis(x, scale=lambdav[it]),
+         xlim=c(-4, 4), type="l", xlab="", ylab="", lwd=4,
+         col=colorv[it], add=(it>1))
+ } ## end for
> # Add title
> title(main="Logistic function", line=0.5)
> # Add legend
> legend("topleft", title="Scale parameters",
+       paste("lambda", lambdav, sep=""), y.intersp=0.4,
+       inset=0.05, cex=0.8, lwd=6, bty="n", lty=1, col=colorv)
```

Performing *Logistic Regression* Using the Function glm()

Logistic regression (logit) is used when the response are discrete variables (like factors or integers), when *linear regression* can't be applied.

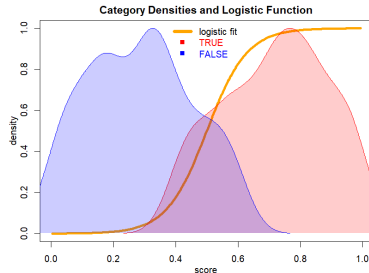
The function `glm()` fits generalized linear models, including *logistic regressions*.

The parameter `family=binomial(logit)` specifies a binomial distribution of residuals in the *logistic regression* model.

The *Mann-Whitney test null hypothesis* is that the two samples, x_i and y_i , were obtained from probability distributions with the same median (location).

The function `wilcox.test()` with parameter `paired=FALSE` (the default) calculates the *Mann-Whitney* test statistic and its p -value.

```
> # Initialize the random number generator
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> # Simulate overlapping scores data
> sample1 <- runif(100, max=0.6)
> sample2 <- runif(100, min=0.4)
> # Perform Mann-Whitney test for data location
> wilcox.test(sample1, sample2)
> # Combine scores and add categorical variable
> predm <- c(sample1, sample2)
> respv <- c(logical(100), !logical(100))
> # Perform logit regression
> logmod <- glm(respv ~ predm, family=binomial(logit))
> class(logmod)
> summary(logmod)
```



```
> ordern <- order(predm)
> plot(x=predm[ordern], y=logmod$fitted.values[ordern],
+      main="Category Densities and Logistic Function",
+      type="l", lwd=4, col="orange", xlab="predictor", ylab="density")
> densv <- density(predm[respv])
> densv$y <- densv$y/max(densv$y)
> lines(densv, col="red")
> polygon(c(min(densv$x), densv$x, max(densv$x)), c(min(densv$y), densv$y, max(densv$y)), col="red")
> densv <- density(predm[!respv])
> densv$y <- densv$y/max(densv$y)
> lines(densv, col="blue")
> polygon(c(min(densv$x), densv$x, max(densv$x)), c(min(densv$y), densv$y, max(densv$y)), col="blue")
> # Add legend
> legend(x="top", cex=1.0, bty="n", lty=c(1, NA, NA),
+       lwd=c(6, NA, NA), pch=c(NA, 15, 15), y.intersp=0.4,
+       legend=c("logistic fit", "TRUE", "FALSE"),
+       col=c("orange", "red", "blue"),
+       text.col=c("black", "red", "blue"))
```

The Likelihood Function of the Binomial Distribution

Let r be a binomial response variable, which either has the value $b = 1$ with probability p , or $b = 0$ with probability $(1 - p)$.

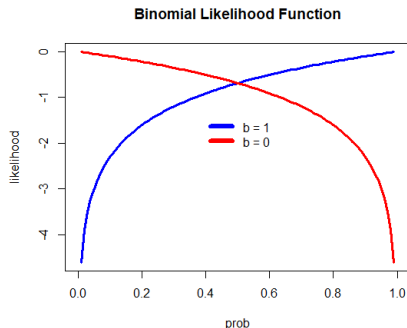
Then the response r follows the binomial distribution:

$$f(b) = b p + (1 - b) (1 - p)$$

The *log-likelihood function* $\mathcal{L}(p|b)$ of the probability p given the value r is obtained from the logarithms of the binomial probabilities:

$$\mathcal{L}(p|b) = b \log(p) + (1 - b) \log(1 - p)$$

The *log-likelihood function* measures how *likely* are the distribution parameters, given the observed values.



```
> # Likelihood function of binomial distribution
> likefun <- function(prob, b) {
+   b*log(prob) + (1-b)*log(1-prob)
+ } ## end likefun
> likefun(prob=0.25, b=1)
> # Plot binomial likelihood function
> curve(expr=likefun(x, b=1), xlim=c(0, 1), lwd=3,
+       xlab="prob", ylab="likelihood", col="blue",
+       main="Binomial Likelihood Function")
> curve(expr=likefun(x, b=0), lwd=3, col="red", add=TRUE)
> legend(x="top", legend=c("b = 1", "b = 0"),
+       title=NULL, inset=0.3, cex=1.0, lwd=6, y.intersp=0.4,
+       bty="n", lty=1, col=c("blue", "red"))
```

The Likelihood Function of the Logistic Model

Let r_i be binomial response variables, with probabilities p_i that depend on the predictor variables s_i through the logistic function:

$$p_i = \frac{1}{1 + \exp(-\lambda_0 - \lambda_1 s_i)}$$

Let's assume that the r_i response and s_i predictor values are known (observed), and we want to find the parameters λ_0 and λ_1 that best fit the observations.

The *log-likelihood function* $\mathcal{L}$ is equal to the sum of the individual *log-likelihoods*:

$$\mathcal{L}(\lambda_0, \lambda_1 | r_i) = \sum_{i=1}^n r_i \log(p_i) + (1 - r_i) \log(1 - p_i)$$

The *log-likelihood function* measures how *likely* are the distribution parameters, given the observed values.

```
> # Add intercept column to the predictor matrix
> predm <- cbind(intercept=rep(1, NROW(respv)), predm)
> # Likelihood function of the logistic model
> likefun <- function(coeff, respv, predm) {
+   probs <- plogis(drop(predm %*% coeff))
+   -sum(respv*log(probs) + (1-respv)*log((1-probs)))
+ } ## end likefun
> # Run likelihood function
> coeff <- c(1, 1)
> likefun(coeff, respv, predm)
```

Multi-dimensional Optimization Using optim()

The function `optim()` performs *multi-dimensional* optimization.

The argument `fn` is the objective function to be minimized.

The argument of `fn` that is to be optimized, must be a vector argument.

The argument `par` is the initial vector argument value.

`optim()` accepts additional parameters bound to the dots `"..."` argument, and passes them to the `fn` objective function.

The arguments `lower` and `upper` specify the search range for the variables of the objective function `fn`.

`method="L-BFGS-B"` specifies the quasi-Newton *gradient* optimization method.

`optim()` returns a list containing the location of the minimum and the objective function value.

The *gradient* methods used by `optim()` can only find the local minimum, not the global minimum.

```
> # Rastrigin function with vector argument for optimization
> rastrigin <- function(vecv, param=25) {
+   sum(vecv^2 - param*cos(vecv))
+ } ## end rastrigin
> vecv <- c(pi/6, pi/6)
> rastrigin(vecv=vecv)
> # Draw 3d surface plot of Rastrigin function
> options(rgl.useNULL=TRUE); library(rgl)
> rgl::persp3d(
+   x=Vectorize(function(x, y) rastrigin(vecv=c(x, y))),
+   xlim=c(-10, 10), ylim=c(-10, 10),
+   col="green", axes=FALSE, zlab="", main="rastrigin")
> # Render the 3d surface plot of function
> rgl::rglwidget(elementId="plot3drgl", width=400, height=400)
> # Optimize with respect to vector argument
> optim1 <- optim(par=vecv, fn=rastrigin,
+   method="L-BFGS-B",
+   upper=c(4*pi, 4*pi),
+   lower=c(pi/2, pi/2),
+   param=1)
> # Optimal parameters and value
> optim1$par
> optim1$value
> rastrigin(optim1$par, param=1)
```

Maximum Likelihood Calibration of the Logistic Model

The logistic model depends on the unknown parameters λ_0 and λ_1 , which can be calibrated by maximizing the likelihood function.

The function `optim()` with the argument `hessian=TRUE` returns the Hessian matrix.

The Hessian is a matrix of the second-order partial derivatives of the likelihood function with respect to the optimization parameters:

$$H = \frac{\partial^2 \mathcal{L}}{\partial \lambda^2}$$

The Hessian matrix measures the convexity of the likelihood surface - it's large if the likelihood surface is highly convex, and it's small if the likelihood surface is flat.

If the likelihood surface is highly convex, then the coefficients can be determined with greater precision, so their standard errors are small. If the likelihood surface is flat, then the coefficients have large standard errors.

The inverse of the Hessian matrix provides the standard errors of the logistic parameters: $\sigma_{SE} = \sqrt{H^{-1}}$.

```
> # Initial parameters
> initp <- c(1, 1)
> # Find max likelihood parameters using steepest descent optimizer
> optim1 <- optim(par=initp,
+   fn=likefun, ## Log-likelihood function
+   method="L-BFGS-B", ## Quasi-Newton method
+   respv=respv,
+   predm=predm,
+   upper=c(20, 20), ## Upper constraint
+   lower=c(-20, -20), ## Lower constraint
+   hessian=TRUE)
> # Optimal logistic parameters
> optim1$par
> unname(logmod$coefficients)
> # Standard errors of parameters
> sqrt(diag(solve(optim1$hessian)))
> regsum <- summary(logmod)
> regsum$coefficients[, 2]
```

Package *ISLR* With Datasets for Machine Learning

The package *ISLR* contains datasets used in the book *Introduction to Statistical Learning* by Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani.

The book introduces machine learning techniques using R, and it's a must for advanced finance applications.

```
> library(ISLR) ## Load package ISLR
> # get documentation for package tseries
> packageDescription("ISLR") ## get short description
>
> help(package="ISLR") ## Load help page
>
> library(ISLR) ## Load package ISLR
>
> data(package="ISLR") ## list all datasets in ISLR
>
> ls("package:ISLR") ## list all objects in ISLR
>
> detach("package:ISLR") ## Remove ISLR from search path
```


The Default Dataset

The data frame `Default` in the package *ISLR* contains credit default data.

The `Default` data frame contains two columns of categorical data (factors): `default` and `student`, and two columns of numerical data: `balance` and `income`.

The columns `default` and `student` contain factor data, and they can be converted to Boolean values, with `TRUE` if `default == "Yes"` and `student == "Yes"`, and `FALSE` otherwise.

This avoids implicit coercion by the function `glm()`.

```
> # Coerce the default and student columns to Boolean
> Default <- ISLR::Default
> Default$default <- (Default$default == "Yes")
> Default$student <- (Default$student == "Yes")
> attach(Default) ## Attach Default to search path
> # Explore credit default data
> summary(Default)
  default      student      balance      income
Mode :logical   Mode :logical   Min.   :    0   Min.   :  772
FALSE:9667      FALSE:7056      1st Qu.: 482   1st Qu.:21340
TRUE :333        TRUE :2944        Median : 824   Median :34553
                                Mean   : 835   Mean   :33517
                                3rd Qu.:1166   3rd Qu.:43808
                                Max.   :2654   Max.   :73554

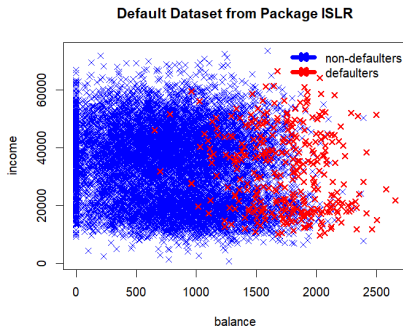
> sapply(Default, class)
  default student balance income
"logical" "logical" "numeric" "numeric"
> dim(Default)
[1] 10000    4
> head(Default)
  default student balance income
1  FALSE  FALSE    730  44362
2  FALSE  TRUE    817  12106
3  FALSE  FALSE   1074  31767
4  FALSE  FALSE    529  35704
5  FALSE  FALSE    786  38463
6  FALSE  TRUE    920   7492
```

The Dependence of default on The balance and income

The columns student, balance, and income can be used as *predictors* to predict the default column.

The scatterplot of income versus balance shows that the balance column is able to separate the data points of default = TRUE from default = FALSE.

But there is very little difference in income between the default = TRUE versus default = FALSE data points.



```
> # Plot data points for non-defaulters
> xlim <- range(balance); ylim <- range(income)
> plot(income ~ balance,
+       main="Default Dataset from Package ISLR",
+       xlim=xlim, ylim=ylim, pch=4, col="blue",
+       data=Default[!default, ])
> # Plot data points for defaulters
> points(income ~ balance, pch=4, lwd=2, col="red",
+        data=Default[default, ])
> # Add legend
> legend(x="topright", legend=c("non-defaulters", "defaulters"),
+        y.intersp=0.4, bty="n", col=c("blue", "red"), lty=1, lwd=6, pch=4)
```

Boxplots of the Default Dataset

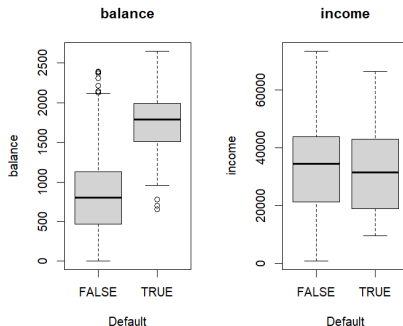
A *Box Plot* (box-and-whisker plot) is a graphical display of a distribution of data:

The *box* represents the upper and lower quartiles, The vertical lines (whiskers) represent values beyond the quartiles, Open circles represent values beyond the nominal range (outliers).

The function `boxplot()` plots a box-and-whisker plot for a distribution of data.

`boxplot()` has two methods: one for formula objects (involving categorical variables), and another for data frames.

The *Mann-Whitney* test shows that the *balance* column provides a strong separation between defaulters and non-defaulters, but the *income* column doesn't.



```
> # Perform Mann-Whitney test for the location of the balances
> wilcox.test(balance[default], balance[!default])
> # Perform Mann-Whitney test for the location of the incomes
> wilcox.test(income[default], income[!default])
> # Plot the densities of the balance amounts for defaulters and non-defaulters
> plot(density(balance[default]),
+      main="Balance Amounts for Defaulters and Non-Defaulters",
+      xlab="balance", ylab="density", col="red", lwd=2)
> lines(density(balance[!default]), col="blue", lwd=2)
> legend(x="topright", inset=0.0, bty="n", lwd=6, y.intersp=0.4,
+       legend=c("non-defaulters", "defaulters"),
+       col=c("blue", "red"))
```

```
> x11(width=6, height=5)
> # Set 2 plot panels
> par(mfrow=c(1,2))
> # Balance boxplot
> boxplot(formula=balance ~ default,
+         col="lightgrey", main="balance", xlab="Default")
> # Income boxplot
> boxplot(formula=income ~ default,
+         col="lightgrey", main="income", xlab="Default")
```

Modeling Credit Defaults Using *Logistic Regression*

The balance column can be used to calculate the probability of default using *logistic regression*.

The residuals are the differences between the actual response values (0 and 1), and the calculated probabilities of default.

The residuals are not normally distributed, so the data is fitted using the *maximum likelihood* method, instead of least squares.

```
> # Fit logistic regression model
> logmod <- glm(default ~ balance, family=binomial(logit))
> class(logmod)
[1] "glm" "lm"
> summary(logmod)
```

```
Call:
glm(formula = default ~ balance, family = binomial(logit))
```

Coefficients:

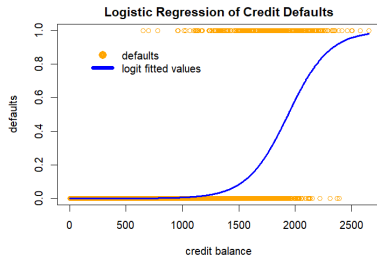
	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-10.65133	0.36116	-29.5	<2e-16 ***
balance	0.00550	0.00022	24.9	<2e-16 ***

```
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

(Dispersion parameter for binomial family taken to be 1)

```
Null deviance: 2920.6  on 9999  degrees of freedom
Residual deviance: 1596.5  on 9998  degrees of freedom
AIC: 1600
```

```
Number of Fisher Scoring iterations: 8
```



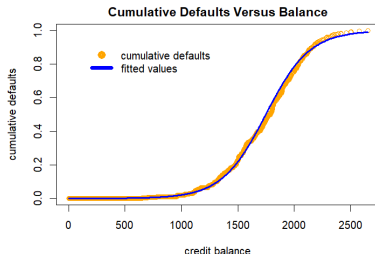
```
> x11(width=6, height=5)
> par(mar=c(4, 4, 2, 2), oma=c(0, 0, 0, 0), mgp=c(2.5, 1, 0))
> plot(x=balance, y=default,
+      main="Logistic Regression of Credit Defaults",
+      col="orange", xlab="credit balance", ylab="defaults")
> ordern <- order(balance)
> lines(x=balance[ordern], y=logmod$fitted.values[ordern], col="blue")
> legend(x="topleft", inset=0.1, bty="n", lwd=6, y.intersp=0.4,
+      legend=c("defaults", "logit fitted values"),
+      col=c("orange", "blue"), lty=c(NA, 1), pch=c(1, NA))
```

Modeling Cumulative Defaults Using *Logistic Regression*

The function `glm()` can model a *logistic* regression using either a Boolean response variable, or using a response variable specified as a frequency.

In the second case, the response variable should be defined as a two-column matrix, with the cumulative frequency of success (TRUE) and a cumulative frequency of failure (FALSE).

These two different ways of specifying the *logistic* regression are related, but they are not equivalent, because they have different error terms.



```
> # Calculate the cumulative defaults
> sumd <- sum(default)
> defaultv <- sapply(balance, function(balv) {
+   sum(default[balance <= balv])
+ }) ## end sapply
> # Perform logit regression
> logmod <- glm(cbind(defaultv, sumd-defaultv) ~ balance,
+   family=binomial(logit))
> summary(logmod)
```

```
> plot(x=balance, y=defaultv/sumd, col="orange", lwd=1,
+   main="Cumulative Defaults Versus Balance",
+   xlab="credit balance", ylab="cumulative defaults")
> ordern <- order(balance)
> lines(x=balance[ordern], y=logmod$fitted.values[ordern],
+   col="blue", lwd=3)
> legend(x="topleft", inset=0.1, bty="n", y.intersp=0.4,
+   legend=c("cumulative defaults", "fitted values"),
+   col=c("orange", "blue"), lty=c(NA, 1), pch=c(1, NA), lwd=6)
```

Multifactor Logistic Regression

Logistic regression calculates the probability of categorical variables, from the *Odds Ratio* of continuous predictors:

$$p = \frac{1}{1 + \exp(-\lambda_0 - \sum_{i=1}^n \lambda_i x_i)}$$

The *generic* function `summary()` produces a list of regression model summary and diagnostic statistics:

- coefficients: matrix with estimated coefficients, their z-values, and p-values,
- *Null* deviance: measures the differences between the response values and the probabilities calculated using only the intercept,
- *Residual* deviance: measures the differences between the response values and the model probabilities,

The *balance* and *student* columns are statistically significant, but the *income* column is not.

```
> # Fit multifactor logistic regression model
> colv <- colnames(Default)
> formulav <- as.formula(paste(colv[1],
+   paste(colv[-1], collapse="+"), sep=" ~ "))
> formulav
default ~ student + balance + income
> logmod <- glm(formulav, data=Default, family=binomial(logit))
> summary(logmod)
```

Call:
glm(formula = formulav, family = binomial(logit), data = Default)

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-1.09e+01	4.92e-01	-22.08	<2e-16 ***
studentTRUE	-6.47e-01	2.36e-01	-2.74	0.0062 **
balance	5.74e-03	2.32e-04	24.74	<2e-16 ***
income	3.03e-06	8.20e-06	0.37	0.7115

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 2920.6 on 9999 degrees of freedom
Residual deviance: 1571.5 on 9996 degrees of freedom
AIC: 1580

Number of Fisher Scoring iterations: 8

Modeling Credit Defaults Using Student Status

The student column alone can be used to calculate the probability of default using a single-factor *logistic* regression.

But the coefficient from the single-factor regression is positive (indicating that students are more likely to default), while the coefficient from the multifactor regression is negative (indicating that students are less likely to default).

The reason that students are more likely to default is because they have higher credit balances than non-students - which is what the single-factor regression shows.

But students are less likely to default than non-students that have the same credit balance - which is what the multifactor model shows.

That's why the multifactor regression coefficient for student is negative, while the single factor coefficient for student is positive.

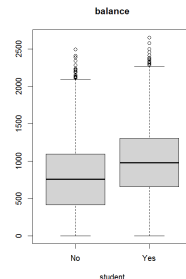
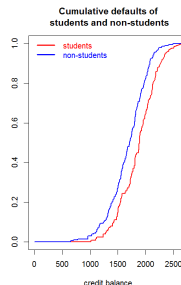
```
> # Fit single-factor logistic model with student as predictor
> studentmod <- glm(default ~ student, family=binomial(logit))
> summary(studentmod)
> # Multifactor coefficient is negative
> logmod$coefficients
> # Single-factor coefficient is positive
> studentmod$coefficients
```

Confounding Variables in Multifactor *Logistic* Regression

The student column is a confounding variable because it's correlated with the balance column.

The plots and boxplots show that students have higher credit balances than non-students.

```
> # Calculate the cumulative defaults
> defcum <- sapply(balance, function(balv) {
+   c(student=sum(default[student & (balance <= balv)]),
+     non_student=sum(default[!student & (balance <= balv)]))
+ }) ## end sapply
> deftotal <- c(student=sum(student & default),
+   student=sum(!student & default))
> defcum <- t(defcum / deftotal)
> # Plot cumulative defaults
> par(mfrow=c(1,2)) ## Set plot panels
> ordern <- order(balance)
> plot(x=balance[ordern], y=defcum[ordern, 1],
+   col="red", t="l", lwd=2, xlab="credit balance", ylab="",
+   main="Cumulative defaults of\n students and non-students")
> lines(x=balance[ordern], y=defcum[ordern, 2], col="blue", lwd=2)
> legend(x="topleft", bty="n", y.intersp=0.4,
+   legend=c("students", "non-students"),
+   col=c("red", "blue"), text.col=c("red", "blue"), lwd=3)
> # Balance boxplot for student factor
> boxplot(formula=balance ~ student,
+   col="lightgrey", main="balance", xlab="Student")
```



Forecasting Credit Defaults using Logistic Regression

The function `predict()` is a *generic function* for forecasting based on a given model.

The method `predict.glm()` produces forecasts for a generalized linear (*glm*) model, in the form of numeric probabilities, not the Boolean response variable.

The Boolean forecasts are obtained by comparing the *forecast probabilities* with a *discrimination threshold*.

Let the *null hypothesis* be that the subject will not default: `default = FALSE`.

If the *forecast probability* is *less* than the *discrimination threshold*, then the forecast is that the subject will not default and that the *null hypothesis* is TRUE.

The *in-sample forecasts* are just the fitted values of the *glm* model.

```
> # Perform in-sample forecast from logistic regression model
> fcast <- predict(logmod, type="response")
> all.equal(logmod$fitted.values, fcast)
[1] TRUE
> # Define discrimination threshold value
> threshv <- 0.7
> # Calculate the confusion matrix in-sample
> table(actual=!default, forecast=(fcast < threshv))
      forecast
actual FALSE TRUE
FALSE    57   276
TRUE     12  9655
> # Fit logistic regression over training data
> # Initialize the random number generator
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> nrows <- NROW(Default)
> samplev <- sample.int(n=nrows, size=nrows/2)
> trainset <- Default[samplev, ]
> logmod <- glm(formulav, data=trainset, family=binomial(logit))
> # Forecast over test data out-of-sample
> testset <- Default[-samplev, ]
> fcast <- predict(logmod, newdata=testset, type="response")
> # Calculate the confusion matrix out-of-sample
> table(actual=!testset$default, forecast=(fcast < threshv))
      forecast
actual FALSE TRUE
FALSE    29   132
TRUE      9  4830
```

Forecasting Errors

A *binary classification model* categorizes cases based on its forecasts whether the *null hypothesis* is TRUE or FALSE.

Let the *null hypothesis* be that the subject will not default: `default = FALSE`.

A *positive* result corresponds to rejecting the null hypothesis, while a *negative* result corresponds to accepting the null hypothesis.

The forecasts are subject to two different types of errors: *type I* and *type II* errors.

A *type I* error is the incorrect rejection of a TRUE *null hypothesis* (i.e. a "false positive"), when there is no default but it's classified as a default.

A *type II* error is the incorrect acceptance of a FALSE *null hypothesis* (i.e. a "false negative"), when there is a default but it's classified as no default.

```
> # Calculate the confusion matrix out-of-sample
> confmat <- table(actual=!testset$default,
+ forecast=(fcast < threshv))
> confmat
      forecast
actual FALSE TRUE
FALSE    29  132
TRUE     9 4830
> # Calculate the FALSE positive (type I error)
> sum(!testset$default & (fcast > threshv))
[1] 9
> # Calculate the FALSE negative (type II error)
> sum(testset$default & (fcast < threshv))
[1] 132
```

The Confusion Matrix of a Binary Classification Model

The confusion matrix summarizes the performance of a classification model on a set of test data for which the actual values of the *null hypothesis* are known.

		Forecast	
		Null is FALSE	Null is TRUE
Actual	Null is FALSE	True Positive (sensitivity)	False Negative (type II error)
	Null is TRUE	False Positive (type I error)	True Negative (specificity)

```
> # Calculate the FALSE positive and FALSE negative rates
> confmat <- confmat / rowSums(confmat)
> c(typeI=confmat[2, 1], typeII=confmat[1, 2])
typeI typeII
0.00186 0.81988
> detach(Default)
```

Let the *null hypothesis* be that the subject will not default: default = FALSE.

The *true positive rate* (known as the *sensitivity*) is the fraction of FALSE *null hypothesis* cases that are correctly classified as FALSE.

The *false negative rate* is the fraction of FALSE *null hypothesis* cases that are incorrectly classified as TRUE (*type II error*).

The sum of the *true positive* plus the *false negative* rate is equal to 1.

The *true negative rate* (known as the *specificity*) is the fraction of TRUE *null hypothesis* cases that are correctly classified as TRUE.

The *false positive rate* is the fraction of TRUE *null hypothesis* cases that are incorrectly classified as FALSE (*type I error*).

The sum of the *true negative* plus the *false positive* rate is equal to 1.

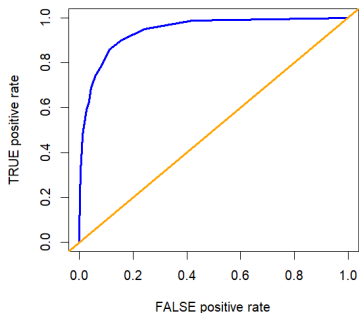
Receiver Operating Characteristic (ROC) Curve

The *ROC curve* is the plot of the *true positive rate*, as a function of the *false positive rate*, and illustrates the performance of a binary classifier.

The area under the *ROC curve* (AUC) is a measure of the performance of a binary classification model.

```
> # Confusion matrix as function of threshold
> confun <- function(actualv, fcast, threshv) {
+   confmat <- table(actualv, (fcast < threshv))
+   confmat <- confmat / rowSums(confmat)
+   c(typeI=confmat[2, 1], typeII=confmat[1, 2])
+ } ## end confun
> confun(!testset$default, fcast, threshv=threshv)
> # Define vector of discrimination thresholds
> threshv <- seq(0.05, 0.95, by=0.05)^2
> # Calculate the error rates
> errorr <- sapply(threshv, confun,
+   actualv=!testset$default, fcast=fcast) ## end sapply
> errorr <- t(errorr)
> rownames(errorr) <- threshv
> errorr <- rbind(c(1, 0), errorr)
> errorr <- rbind(errorr, c(0, 1))
> # Calculate the area under ROC curve (AUC)
> truepos <- (1 - errorr[, "typeII"])
> truepos <- (truepos + rutils::lagit(truepos))/2
> falsepos <- rutils::diffit(errorr[, "typeI"])
> abs(sum(truepos*falsepos))
```

ROC Curve for Defaults



```
> # Plot ROC Curve for Defaults
> x11(width=5, height=5)
> plot(x=errorr[, "typeI"], y=1-errorr[, "typeII"],
+   xlab="FALSE positive rate", ylab="TRUE positive rate",
+   main="ROC Curve for Defaults", type="l", lwd=3, col="blue")
> abline(a=0.0, b=1.0, lwd=3, col="orange")
```

Types of Bad Data

Possible sources of bad data are: imported data, class coercion, numeric overflow.

Types of bad data:

- NA (not available) is a logical constant indicating missing data,
- NaN means Not a Number data,
- Inf means numeric overflow - divide by zero,

When a function produces NA or NaN values, then it also produces a *warning* condition, but not an *error*.

NA or NaN values are not *errors*.

The functions `is.na()` and `is.nan()` test for NA and NaN values.

Many functions have a `na.rm` parameter to remove NAs from input data.

```
> as.numeric(c(1:3, "a")) ## NA from coercion
[1] 1 2 3 NA
> 0/0 ## NaN from ambiguous math
[1] NaN
> 1/0 ## Inf from divide by zero
[1] Inf
> is.na(c(NA, NaN, 0/0, 1/0)) ## Test for NA
[1] TRUE TRUE TRUE FALSE
> is.nan(c(NA, NaN, 0/0, 1/0)) ## Test for NaN
[1] FALSE TRUE TRUE FALSE
> NA*1:4 ## Create vector of NAs
[1] NA NA NA NA
> # Create vector with some NA values
> datav <- c(1, 2, NA, 4, NA, 5)
> datav
[1] 1 2 NA 4 NA 5
> mean(datav) ## Returns NA, when NAs are input
[1] NA
> mean(datav, na.rm=TRUE) ## remove NAs from input data
[1] 3
> datav[!is.na(datav)] ## Delete the NA values
[1] 1 2 4 5
> sum(!is.na(datav)) ## Count non-NA values
[1] 4
```

Scrubbing Bad Data

The function `complete.cases()` returns TRUE if a row has no NA values.

```
> # airquality data has some NAs
> head(airquality)
  Ozone Solar.R Wind Temp Month Day
1   41    190  7.4   67     5   1
2   36    118  8.0   72     5   2
3   12    149 12.6   74     5   3
4   18    313 11.5   62     5   4
5   NA     NA 14.3   56     5   5
6   28     NA 14.9   66     5   6

> dim(airquality)
[1] 153  6

> # Number of NA elements
> sum(is.na(airquality))
[1] 44

> # Number of rows with NA elements
> sum(!complete.cases(airquality))
[1] 42

> # Display rows containing NAs
> head(airquality[!complete.cases(airquality), ])
  Ozone Solar.R Wind Temp Month Day
5     NA     NA 14.3   56     5   5
6     28     NA 14.9   66     5   6
10    NA    194  8.6   69     5  10
11     7     NA  6.9   74     5  11
25    NA     66 16.6   57     5  25
26    NA    266 14.9   58     5  26
```

Scrubbing Data Using Carry Forward

Rows containing bad data may be either removed or replaced with an estimated value.

The function `stats::na.omit()` removes individual NA values from vectors, and it also removes whole rows of data containing NA values from matrices and data frames.

Bad data can also be replaced with the most recent prior values (carry forward good data).

The function `zoo::na.locf()` replaces NA values with the most recent non-NA values prior to it (*lo*cf stands for *last observation carry forward*).

Copying the last non-NA values forward causes less data loss than removing whole rows of data.

The function `na.locf()` with argument `fromLast=TRUE` replaces NA values with non-NA values in reverse order, starting from the end.

```
> # Create vector containing NA values
> vecv <- sample(22)
> vecv[sample(NROW(vecv), 4)] <- NA
> # Replace NA values with the most recent non-NA values
> zoo::na.locf(vecv)
[1] 7 7 7 6 6 18 15 3 9 16 16 12 5 17 8 21 13 1 22 20 2
> # Remove rows containing NAs
> goodair <- airquality[complete.cases(airquality), ]
> dim(goodair)
[1] 111 6
> # NAs removed
> head(goodair)
  Ozone Solar.R Wind Temp Month Day
1    41    190   7.4   67     5   1
2    36    118   8.0   72     5   2
3    12    149  12.6   74     5   3
4    18    313  11.5   62     5   4
7    23    299   8.6   65     5   7
8    19     99  13.8   59     5   8
> # Another way of removing NAs
> freshair <- na.omit(airquality)
> all.equal(freshair, goodair, check.attributes=FALSE)
[1] TRUE
> # Replace NAs
> goodair <- zoo::na.locf(airquality)
> dim(goodair)
[1] 153 6
> # NAs replaced
> head(goodair)
  Ozone Solar.R Wind Temp Month Day
1    41    190   7.4   67     5   1
2    36    118   8.0   72     5   2
3    12    149  12.6   74     5   3
4    18    313  11.5   62     5   4
5    18    313  14.3   56     5   5
6    28    313  14.9   66     5   6
```

Scrubbing Time Series Data

Missing asset prices and returns can be replaced with the most recent prior values (carry forward good data).

But missing asset returns should not be replaced with values from the future. Instead, missing returns should be replaced with zero values.

The function `na.locf.xts()` from package `xts` is faster than `zoo::na.locf()`, but it only operates on time series of class "xts".

```
> # Replace NAs in xts time series
> library(rutils) ## load package rutils
> pricev <- rutils::etfenv$prices[, 1]
> head(pricev, 3)
           VEU
1993-01-29  NA
1993-02-01  NA
1993-02-02  NA
> sum(is.na(pricev))
[1] 3552
> pricez <- zoo::na.locf(pricev, fromLast=TRUE)
> pricex <- xts::na.locf.xts(pricev, fromLast=TRUE)
> all.equal(pricez, pricex, check.attributes=FALSE)
[1] TRUE
> head(pricex, 3)
           VEU
1993-01-29 30.4
1993-02-01 30.4
1993-02-02 30.4
> library(microbenchmark)
> summary(microbenchmark(
+   zoo=zoo::na.locf(pricev, fromLast=TRUE),
+   xts=xts::na.locf.xts(pricev, fromLast=TRUE),
+   times=10))[, c(1, 4, 5)] ## end microbenchmark summary
      expr mean median
1  zoo 29.9   28.3
2  xts 27.5   27.4
```


NULL Values

NULL represents a null object, and is a legitimate value, not bad data.

NULL is often returned by functions whose value is undefined.

NULL can also be used to initialize vectors.

NULL is not the same as NA values or zero-length (empty) vectors.

The functions `numeric()` and `character()` return empty (zero-length) vectors of the specified *type*.

The function `is.null()` tests for NULL values.

Very often variables are initialized to NULL before the start of iteration.

A more efficient way to perform iteration is by pre-allocating the vector.

```
> # NULL values have no mode or type
> c(mode(NULL), mode(NA))
[1] "NULL"      "logical"
> c(typeof(NULL), typeof(NA))
[1] "NULL"      "logical"
> c(NROW(NULL), NROW(NA))
[1] 0 1
> # Check for NULL values
> is.null(NULL)
[1] TRUE
> # NULL values are ignored when combined into a vector
> c(1, 2, NULL, 4, 5)
[1] 1 2 4 5
> # But NA value isn't ignored
> c(1, 2, NA, 4, 5)
[1] 1 2 NA 4 5
> # Vectors can be initialized to NULL
> vecv <- NULL
> is.null(vecv)
[1] TRUE
> # Grow the vector in a loop - very bad code!!!
> for (indeks in 1:5)
+   vecv <- c(vecv, indeks)
> # Initialize empty vector
> vecv <- numeric()
> # Grow the vector in a loop - very bad code!!!
> for (indeks in 1:5)
+   vecv <- c(vecv, indeks)
> # Allocate vector
> vecv <- numeric(5)
> # Assign to vector in a loop - good code
> for (indeks in 1:5)
+   vecv[indeks] <- runif(1)
```

Scrubbing Isolated Spikes In Stock Prices

Isolated price spikes can be identified using a *three-point filter* (tri-filter).

The *z-scores* are equal to the prices minus the mean of the neighboring prices, divided by their differences:

$$z_i = \frac{p_i - 0.5(p_{i-1} + p_{i+1}))}{p_{i-1} - p_{i+1}}$$

If the absolute value of the *z-score* exceeds the *threshold value* then it's classified as *bad data*, and it can be removed or replaced with the previous price.

The *tri-filter* can only remove isolated price spikes, but not consecutive price spikes.

XLK Intraday Prices for 2020-03-16



```
> # Load and plot intraday stock prices
> load("/Users/jerzy/Develop/lecture_slides/data/xlk_tick_trades_2020-03-16")
> pricev <- xlk$price
> dygraphs::dygraph(pricev, main="XLK Intraday Prices for 2020-03-16")
+   dyOptions(colors="blue", strokeWidth=1)
> # Calculate the lagged and advanced prices
> pricelag <- rutils::lagit(pricev)
> pricelag[1] <- pricelag[2]
> pricadv <- rutils::lagit(pricev, lag=-1)
> pricadv[NROW(pricadv)] <- pricadv[NROW(pricadv)-1]
> # Calculate the z-scores
> diff1 <- ifelse(abs(pricelag-pricadv) < 0.01, 0.01, abs(pricelag-pricadv))
> zscores <- (pricev - 0.5*(pricelag+pricadv))/diff1
```

```
> # Z-scores have very fat tails
> range(zscores); mad(zscores)
> madz <- mad(zscores[abs(zscores) > 0])
> hist(zscores, breaks=5000, xlim=c(-2*madz, 2*madz))
> # Scrub the price spikes
> threshv <- 5*madz ## Discrimination threshold
> indeks <- which(abs(zscores) > threshv)
> pricev[indeks] <- as.numeric(pricev[indeks-1])
> # Plot dygraph of the scrubbed prices
> dygraphs::dygraph(pricev, main="Scrubbed XLK Intraday Prices")
+   dyOptions(colors="blue", strokeWidth=1)
```

The Hampel Filter For Filtering Price Spikes

Price spikes in high frequency data can be identified using a *Hampel filter*.

The z-scores are equal to the prices minus the median of the prices, divided by the median absolute deviation (*MAD*) of prices:

$$z_i = \frac{p_i - \text{median}(p)}{\text{MAD}}$$

If the absolute value of the z-score exceeds the *threshold value* then it's classified as *bad data*, and it can be removed or replaced.

```
> # Calculate the centered Hampel filter to remove bad prices
> lookb <- 71 ## Look-back interval
> halfb <- lookb %/% 2 ## Half-back interval
> pricev <- xlk$price
> # Calculate the trailing median and MAD
> medianv <- HighFreq::roll_mean(pricev, lookb=lookb, method="nonp
> colnames(medianv) <- c("median")
> madv <- HighFreq::roll_var(pricev, lookb=lookb, method="nonparam
> # madv <- TTR::runMAD(pricev, n=lookb)
> # Center the median and the MAD
> medianv <- rutils::lagit(medianv, lagg=(-halfb), pad_zeros=FALSE)
> madv <- rutils::lagit(madv, lagg=(-halfb), pad_zeros=FALSE)
> # Calculate the Z-scores
> zscores <- ifelse(madv > 0, (pricev - medianv)/madv, 0)
> # Z-scores have very fat tails
> range(zscores); mad(zscores)
> madz <- mad(zscores[abs(zscores) > 0])
> hist(zscores, breaks=5000, xlim=c(-2*madz, 2*madz))
```

Scrubbed XLK Intraday Prices for 2020-03-16



```
> # Define discrimination threshold value
> threshv <- 6*madz
> # Identify good prices with small z-scores
> isgood <- (abs(zscores) < threshv)
> # Calculate the number of bad prices
> sum(!isgood)
> # Overwrite bad prices and calculate time series of scrubbed price
> pricec <- pricev
> pricec[!isgood] <- NA
> pricec <- zoo::na.locf(pricec)
> # Plot dygraph of the scrubbed prices
> dygraphs::dygraph(pricec, main="Scrubbed XLK Intraday Prices") %>
+   dyOptions(colors="blue", strokeWidth=1)
> # Plot using chart_Series()
> x11(width=6, height=5)
> quantmod::chart_Series(x=pricec,
+   name="Clean XLK Intraday Prices for 2020-03-16")
```

Classifying Data Outliers Using the Hampel Filter

The Hampel filter is a *classifier* which classifies the prices as either good or bad data points.

In order to measure the performance of the Hampel filter, we add price spikes to the clean prices, to see how accurately they're classified.

Let the *null hypothesis* be that the given price is a good data point.

A positive result corresponds to rejecting the *null hypothesis*, while a negative result corresponds to accepting the *null hypothesis*.

The classifications are subject to two different types of errors: *type I* and *type II* errors.

A *type I* error is the incorrect rejection of a TRUE *null hypothesis* (i.e. a "false positive"), when good data is classified as bad.

A *type II* error is the incorrect acceptance of a FALSE *null hypothesis* (i.e. a "false negative"), when bad data is classified as good.

```
> # Add 200 random price spikes to the clean prices
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> nspikes <- 200
> nrows <- NROW(pricex)
> ispike <- logical(nrows)
> ispike[sample(x=nrows, size=nspikes)] <- TRUE
> priceb <- pricex
> priceb[ispike] <- priceb[ispike]*
+   sample(c(0.999, 1.001), size=nspikes, replace=TRUE)
> # Plot the bad prices and their medians
> medianv <- HighFreq::roll_mean(priceb, lookb=lookb, method="nonparametric")
> pricem <- cbind(priceb, medianv)
> colnames(pricem) <- c("prices with spikes", "median")
> dygraphs::dygraph(pricem, main="XLK Prices With Spikes") %>%
+   dyOptions(colors=c("red", "blue"))
> # Calculate the z-scores
> madv <- HighFreq::roll_var(priceb, lookb=lookb, method="nonparametric")
> zscores <- ifelse(madv > 0, (priceb - medianv)/madv, 0)
> # Z-scores have very fat tails
> range(zscores); mad(zscores)
> madz <- mad(zscores[abs(zscores) > 0])
> hist(zscores, breaks=10000, xlim=c(-4*madz, 4*madz))
> # Identify good prices with small z-scores
> threshv <- 3*madz
> isgood <- (abs(zscores) < threshv)
> # Calculate the number of bad prices
> sum(!isgood)
```

Confusion Matrix of a Binary Classification Model

A *binary classification model* categorizes cases based on its forecasts whether the *null hypothesis* is TRUE or FALSE.

The confusion matrix summarizes the performance of a classification model on a set of test data for which the actual values of the *null hypothesis* are known.

		Forecast	
		Null is FALSE	Null is TRUE
Actual	Null is FALSE	True Positive (sensitivity)	False Negative (type II error)
	Null is TRUE	False Positive (type I error)	True Negative (specificity)

```
> # Calculate the confusion matrix
> table(actual=!ispike, forecast=isgood)
> sum(!isgood)
> # FALSE positive (type I error)
> sum(!ispike & !isgood)
> # FALSE negative (type II error)
> sum(ispike & isgood)
```

Let the *null hypothesis* be that the given price is a good data point.

The *true positive rate* (known as the *sensitivity*) is the fraction of FALSE *null hypothesis* cases that are correctly classified as FALSE.

The *false negative rate* is the fraction of FALSE *null hypothesis* cases that are incorrectly classified as TRUE (*type II error*).

The sum of the *true positive* plus the *false negative* rate is equal to 1.

The *true negative rate* (known as the *specificity*) is the fraction of TRUE *null hypothesis* cases that are correctly classified as TRUE.

The *false positive rate* is the fraction of TRUE *null hypothesis* cases that are incorrectly classified as FALSE (*type I error*).

The sum of the *true negative* plus the *false positive* rate is equal to 1.

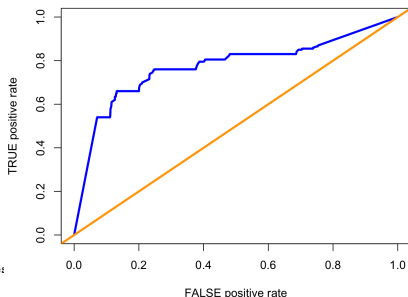
Receiver Operating Characteristic (ROC) Curve

The *ROC curve* is the plot of the *true positive rate*, as a function of the *false positive rate*, and illustrates the performance of a binary classifier.

The area under the *ROC curve* (AUC) measures the classification ability of a binary classifier.

```
> # Confusion matrix as function of threshold
> confun <- function(actualv, zscores, threshv) {
+   confmat <- table(actualv, (abs(zscores) < threshv))
+   confmat <- confmat / rowSums(confmat)
+   c(typeI=confmat[2, 1], typeII=confmat[1, 2])
+ } ## end confun
> confun(!ispike, zscores, threshv=threshv)
> # Define vector of discrimination thresholds
> threshv <- madz*seq(from=0.1, to=3.0, by=0.05)/2
> # Calculate the error rates
> errorr <- sapply(threshv, confun, actualv=!ispike, zscores=zscores)
> errorr <- t(errorr)
> rownames(errorr) <- threshv
> errorr <- rbind(c(1, 0), errorr)
> errorr <- rbind(errorr, c(0, 1))
> # Calculate the area under the ROC curve (AUC)
> truepos <- (1 - errorr[, "typeII"])
> truepos <- (truepos + rutils::lagit(truepos))/2
> falsepos <- rutils::diffit(errorr[, "typeI"])
> abs(sum(truepos*falsepos))
```

ROC Curve for Hampel Classifier



```
> # Plot ROC curve for Hampel classifier
> plot(x=errorr[, "typeI"], y=1-errorr[, "typeII"],
+      xlab="FALSE positive rate", ylab="TRUE positive rate",
+      xlim=c(0, 1), ylim=c(0, 1),
+      main="ROC Curve for Hampel Classifier",
+      type="l", lwd=3, col="blue")
> abline(a=0.0, b=1.0, lwd=3, col="orange")
```

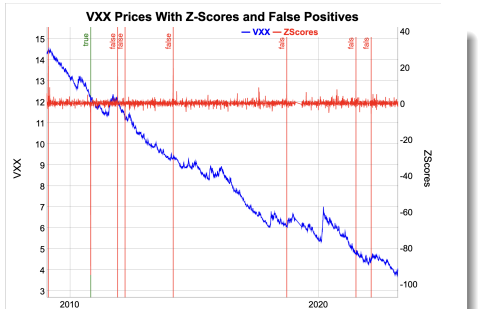
Filtering Bad Data From Daily Stock Prices

Daily stock prices can also contain bad data points consisting of mostly single, isolated spikes in prices.

The number of false positives may be too high, so the Hampel filter parameters (the look-back interval and the threshold) need adjustment.

For example, the VXX has only one bad price (on 2010-11-08), but the Hampel filter identifies many more than that (which are false positives).

```
> # Load log VXX prices
> load("/Users/jerzy/Develop/lecture_slides/data/pricevxx.RData")
> nrows <- NROW(pricev)
> # Calculate the centered Hampel filter for VXX
> lookb <- 7 ## Look-back interval
> halfb <- lookb %/% 2 ## Half-back interval
> medianv <- HighFreq::roll_mean(pricev, lookb=lookb, method="nonparam")
> medianv <- rutils::lagit(medianv, lagg=(-halfb), pad_zeros=FALSE)
> madv <- HighFreq::roll_var(pricev, lookb=lookb, method="nonparam")
> madv <- rutils::lagit(madv, lagg=(-halfb), pad_zeros=FALSE)
> zscores <- ifelse(madv > 0, (pricev - medianv)/madv, 0)
> range(zscores); mad(zscores)
> madz <- mad(zscores[abs(zscores) > 0])
> hist(zscores, breaks=100, xlim=c(-3*madz, 3*madz))
> # Define discrimination threshold value
> threshv <- 9*madz
> # Calculate the good prices
> isgood <- (abs(zscores) < threshv)
> sum(!isgood)
> # Dates of the bad prices
> zoo::index(pricev[!isgood])
```



```
> # Calculate the false positives
> falsep <- !isgood
> falsep[which(zoo::index(pricev) == as.Date("2010-11-08"))] <- FALSE
> # Plot dygraph of the prices with bad prices
> datam <- cbind(pricev, zscores)
> colnames(datam)[2] <- "ZScores"
> colv <- colnames(datam)
> dygraphs::dygraph(datam, main="VXX Prices With Z-Scores and False Positives")
+ dyAxis("y", label=colv[1], independentTicks=TRUE) %>%
+ dyAxis("y2", label=colv[2], independentTicks=TRUE) %>%
+ dySeries(name=colv[1], axis="y", strokeWidth=1, col="blue") %>%
+ dySeries(name=colv[2], axis="y2", strokeWidth=1, col="red") %>%
+ dyEvent(zoo::index(pricev[falsep]), label=rep("false", sum(falsep)))
+ dyEvent(zoo::index(pricev["2010-11-08"]), label="true", stroke="green")
```

Scrubbing Bad Stock Prices

Bad stock prices can be scrubbed (replaced) with the previous good price.

But it's incorrect to replace bad prices with the average of the previous good price and the next good price, since that would cause data snooping.

```
> # Replace bad stock prices with the previous good prices
> priceg <- pricev
> priceg[!isgood] <- NA
> priceg <- zoo::na.locf(priceg)
> # Calculate the Z-scores
> medianv <- HighFreq::roll_mean(priceg, lookb=lookb, method="nonparametric")
> medianv <- rutils::lagit(medianv, lagg=(-halfb), pad_zeros=FALSE)
> madv <- HighFreq::roll_var(priceg, lookb=lookb, method="nonparametric")
> madv <- rutils::lagit(madv, lagg=(-halfb), pad_zeros=FALSE)
> zscores <- ifelse(madv > 0, (priceg - medianv)/madv, 0)
> madz <- mad(zscores[abs(zscores) > 0])
> # Calculate the number of bad prices
> threshv <- 9*madz
> isgood <- (abs(zscores) < threshv)
> sum(!isgood)
> zoo::index(priceg[!isgood])
```

Scrubbed VXX Prices With False Positives



```
> # Calculate the false positives
> falsep <- !isgood
> falsep[which(zoo::index(pricev) == as.Date("2010-11-08"))] <- FALSE
> # Plot dygraph of the prices with bad prices
> dygraphs::dygraph(priceg, main="Scrubbed VXX Prices With False Positives")
+   dyEvent(zoo::index(priceg[falsep]), label=rep("false", sum(falsep)))
+   dyOptions(colors="blue", strokeWidth=1)
```


ROC Curve for Daily Hampel Classifier

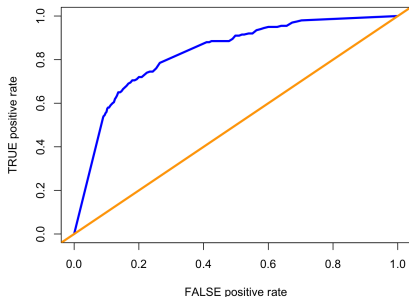
In order to measure the performance of the Hampel filter, we add price spikes to the clean prices, to see how accurately they're classified.

The performance of the Hampel noise classification model depends on the length of the look-back time interval.

The optimal *look-back interval* and *threshold value* can be determined using *cross-validation*.

```
> # Add 200 random price spikes to the clean prices
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> nspikes <- 200
> ispike <- logical(nrows)
> ispike[sample(x=nrows, size=nspikes)] <- TRUE
> priceb <- priceg
> priceb[ispike] <- priceb[ispike]*
+   sample(c(0.99, 1.01), size=nspikes, replace=TRUE)
> # Calculate the Z-scores
> medianv <- HighFreq::roll_mean(priceb, lookb=lookb, method="nonparametric")
> medianv <- rutils::lagit(medianv, lagg=(-halfb), pad_zeros=FALSE)
> madv <- HighFreq::roll_var(priceb, lookb=lookb, method="nonparametric")
> madv <- rutils::lagit(madv, lagg=(-halfb), pad_zeros=FALSE)
> zscores <- ifelse(madv > 0, (priceb - medianv)/madv, 0)
> madz <- mad(zscores[abs(zscores) > 0])
> # Define vector of discrimination thresholds
> threshv <- madz*seq(from=0.1, to=3.0, by=0.05)/2
> # Calculate the error rates
> errorrr <- sapply(threshv, confun, actualv=!ispike, zscores=zscores)
> errorrr <- t(errorrr)
> rownames(errorrr) <- threshv
> errorrr <- rbind(c(1, 0), errorrr)
> errorrr <- rbind(errorrr, c(0, 1))
```

ROC Curve for Daily Hampel Classifier



```
> # Calculate the area under the ROC curve (AUC)
> truepos <- (1 - errorrr[, "typeII"])
> truepos <- (truepos + rutils::lagit(truepos))/2
> falsepos <- rutils::diffit(errorrr[, "typeI"])
> abs(sum(truepos*falsepos))
> # Plot ROC curve for Hampel classifier
> plot(x=errorrr[, "typeI"], y=1-errorrr[, "typeII"],
+      xlab="FALSE positive rate", ylab="TRUE positive rate",
+      xlim=c(0, 1), ylim=c(0, 1),
+      main="ROC Curve for Daily Hampel Classifier",
+      type="l", lwd=3, col="blue")
> abline(a=0.0, b=1.0, lwd=3, col="orange")
```

Brownian Motion

Brownian motion B_T is a stochastic process, with the increments dB_t which are independent and normally distributed, with mean zero and variance equal to dt .

$$dB_t = \xi_t \sqrt{dt}$$

Where the ξ_t are random and independent *innovations* following the standard normal distribution $\phi(0, 1)$, with the expected values: $\mathbb{E}[\xi_t] = 0$, $\mathbb{E}[\xi_t^2] = 1$, and $\mathbb{E}[\xi_{t1}\xi_{t2}] = 0$.

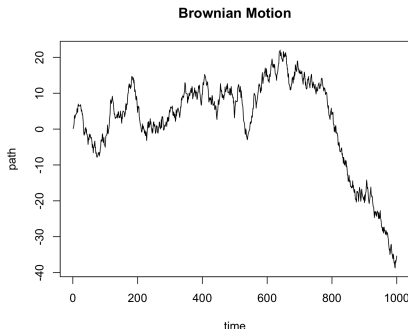
The Brownian motion path B_T is equal to the sum of its increments dB_t :

$$B_T = \sum_{i=1}^n dB_t$$

Where the number of time increments n is equal to the total time of evolution T divided by the increment size dt : $n = T/dt$.

The variance of Brownian motion is equal to the time of its evolution T :

$$\sigma^2 = \mathbb{E}[(\sum_{i=1}^n \xi_i \sqrt{dt})^2] = \sum_{i=1}^n \mathbb{E}[\xi_i^2] dt = T$$



```
> # Simulate a Brownian motion path
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> pathv <- cumsum(rnorm(nrows))
> plot(pathv, type="l", xlab="time", ylab="path",
+       main="Brownian Motion")
```

The Maximum Value of Brownian Motion

The distribution of the maximum value m of a Brownian motion path B_t can be calculated using the *reflection principle*.

The *reflection principle* states that the mirror image (reflection) of a Brownian motion path has the same probability as the original path.

The probability that the Brownian motion path B_t is at some point greater than some value m is the sum of the joint probability, that after the Brownian motion reaches the value m , it ends up greater than m , plus the joint probability that it ends up less than m :

$$p(B_t > m) = p((B_t > m) \& (B_T > m)) + p((B_t > m) \& (B_T < m))$$

By the *reflection principle*, both probabilities are equal, and also $p((B_t > m) \& (B_T > m)) = p(B_T > m)$, so that:

$$p(B_t > m) = 2p(B_T > m)$$

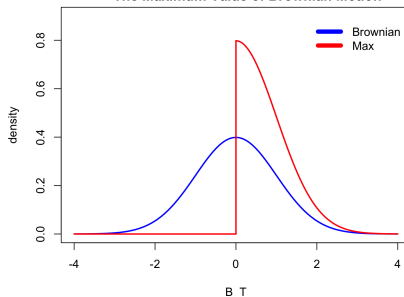
And the probability density of the maximum value m is equal to:

$$\varphi(m) = \sqrt{\frac{2}{\pi T}} e^{-\frac{m^2}{2T}}$$

The average value of the maximum value is equal to:

$$\bar{m} = \sqrt{\frac{2T}{\pi}}$$

Probability Density of
The Maximum Value of Brownian Motion



```
> # Plot the density of Brownian Motion
> curve(expr=dnorm(x), xlim=c(-4, 4), ylim=c(0, 0.9),
+       xlab="B_T", ylab="density", lwd=2, col="blue")
> # Plot the density of the maximum of Brownian Motion
> curve(expr=2*dnorm(x), xlim=c(0, 4), xlab="", ylab="",
+       lwd=2, col="red", add=TRUE)
> lines(x=c(0, 0), y=c(0, sqrt(2/pi)), lwd=2, col="red")
> lines(x=c(-4, 0), y=c(0, 0), lwd=2, col="red")
> title(main="Probability Density of
+ The Maximum Value of Brownian Motion", line=0.5)
> legend("topright", inset=0.0, bty="n", y.intersp=0.4,
+       title=NULL, c("Brownian", "Max"), lwd=6,
+       col=c("blue", "red"))
```

The Range of Brownian Motion

The range of a Brownian motion path B_t is equal to the difference between its maximum value minus its minimum value. The range is also called the drawdown.

The probability density of the range r is equal to the infinite series:

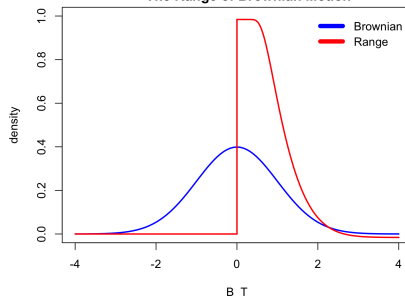
$$p(r) = 2 \sum_{n=1}^{\infty} \frac{\sin(n-0.5)\pi}{(n-0.5)\pi} \left(1 - e^{-\frac{(n-0.5)^2 \pi^2 T}{2r^2}}\right)$$

The average value of the range is equal to:

$$\bar{r} = \sqrt{\frac{\pi T}{2}}$$

```
> # Series element
> fun1 <- function(n, r) { 2*sin((n-0.5)*pi)/((n-0.5)*pi) *
+   (1-exp(-((n-0.5)^2*pi^2/(2*r^2))) }
> # fun2 <- function(x) { sum(sapply(1:10, function(n) fun1(n, x))) }
> # fun2 <- function(x) { fun1(1, x) + fun1(2, x) + fun1(3, x) + fi
> # Sum of fun1
> fun2 <- function(x) {
+   valf <- 0
+   for (n in 1:20) { valf <- valf + fun1(n, x) }
+   return(valf)
+ } ## end fun2
> # Theoretical average value of the range
> fun2(2)
> # Average value of the range from integration (not quite close)
> integrate(fun2, lower=0.01, upper=4)
```

Probability Density of
The Range of Brownian Motion



```
> # Plot the density of Brownian Motion
> curve(expr=dnorm(x), xlim=c(-4, 4), ylim=c(0, 1.0),
+   xlab="B_T", ylab="density", lwd=2, col="blue")
> # Plot the density of the range of Brownian Motion
> curve(expr=fun2(x), xlim=c(0, 4), xlab="", ylab="",
+   lwd=2, col="red", add=TRUE)
> lines(x=c(0, 0), y=c(0, fun2(0.01)), lwd=2, col="red")
> lines(x=c(-4, 0), y=c(0, 0), lwd=2, col="red")
> title(main="Probability Density of
+ The Range of Brownian Motion", line=0.5)
> legend("topright", inset=0.0, bty="n", y.intersp=0.7,
+   title=NULL, c("Brownian", "Range"), lwd=6,
+   col=c("blue", "red"))
```

Geometric Brownian Motion

If the percentage asset returns $r_t dt = d \log p_t$ follow *Brownian motion*:

$$r_t dt = d \log p_t = \left(\mu - \frac{\sigma^2}{2}\right) dt + \sigma dB_t$$

Then asset prices p_t follow *Geometric Brownian motion* (GBM):

$$dp_t = \mu p_t dt + \sigma p_t dB_t$$

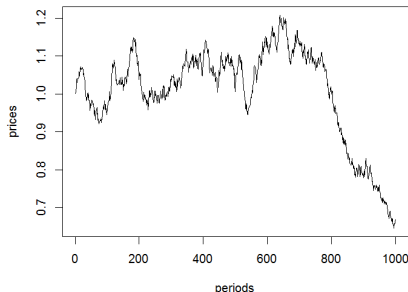
Where σ is the volatility of asset returns, and B_t is a *Brownian Motion*, with dB_t following the normal distribution $\phi(0, \sqrt{dt})$, with the volatility $\sqrt{dt}$, equal to the square root of the time increment dt .

The solution of *Geometric Brownian motion* is equal to:

$$p_t = p_0 \exp\left[\left(\mu - \frac{\sigma^2}{2}\right)t + \sigma B_t\right]$$

The convexity correction: $-\frac{\sigma^2}{2}$ ensures that the growth rate of prices is equal to μ , (according to Ito's lemma).

geometric Brownian motion



```
> # Define the daily volatility and growth rate
> sigmav <- 0.01; drift <- 0.0; nrows <- 1000
> # Simulate geometric Brownian motion
> retp <- sigmav*rnorm(nrows) + drift - sigmav^2/2
> pricev <- exp(cumsum(retp))
> plot(pricev, type="l", xlab="time", ylab="prices",
+      main="Geometric Brownian Motion")
```

Simulating Random OHLC Prices

Random OHLC prices are useful for testing financial models.

The function `sample()` selects a random sample from a vector of data elements.

The function `sample()` with `replace=TRUE` selects samples with replacement (the default is `replace=FALSE`).

```
> # Simulate geometric Brownian motion
> sigmav <- 0.01/sqrt(48)
> drift <- 0.0
> nrows <- 1e4
> datev <- seq(from=as.POSIXct(paste(Sys.Date()-250, "09:30:00")),
+   length.out=nrows, by="30 min")
> pricev <- exp(cumsum(sigmav*rnorm(nrows) + drift - sigmav^2/2))
> pricev <- xts(pricev, order.by=datev)
> pricev <- cbind(pricev,
+   volume=sample(x=10*(2:18), size=nrows, replace=TRUE))
> # Aggregate to daily OHLC data
> ohlc <- xts::to.daily(pricev)
> quantmod::chart_Series(ohlc, name="random prices")
> # dygraphs candlestick plot using pipes syntax
> library(dygraphs)
> dygraphs::dygraph(ohlc[, 1:4]) %>% dyCandlestick()
> # dygraphs candlestick plot without using pipes syntax
> dygraphs::dyCandlestick(dygraphs::dygraph(ohlc[, 1:4]))
```



The Log-normal Probability Distribution

If x follows the *Normal* distribution $\phi(x, \mu, \sigma)$, then the exponential of x : $y = e^x$ follows the *Log-normal* distribution $\log \phi(\cdot)$:

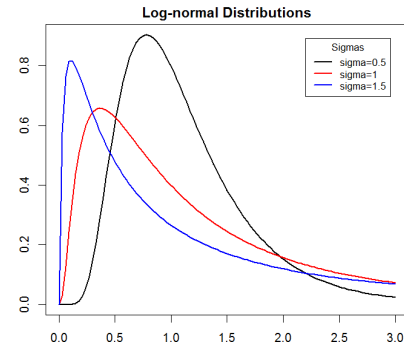
$$\log \phi(y, \mu, \sigma) = \frac{\exp(-(\log y - \mu)^2 / 2\sigma^2)}{y\sigma\sqrt{2\pi}}$$

With mean equal to: $\bar{y} = \mathbb{E}[y] = e^{(\mu + \sigma^2/2)}$, and median equal to: $\tilde{y} = e^\mu$

With variance equal to: $\sigma_y^2 = (e^{\sigma^2} - 1)e^{(2\mu + \sigma^2)}$, and skewness (third moment) equal to:

$$\varsigma = \mathbb{E}[(y - \mathbb{E}[y])^3] = (e^{\sigma^2} + 2)\sqrt{e^{\sigma^2} - 1}$$

```
> # Standard deviations of log-normal distribution
> sigmavs <- c(0.5, 1, 1.5)
> # Create plot colors
> colorv <- c("black", "red", "blue")
> # Plot all curves
> for (indeks in 1:NROW(sigmavs)) {
+   curve(expr=dlnorm(x, sdlog=sigmavs[indeks]),
+         type="l", lwd=2, xlim=c(0, 3),
+         xlab="", ylab="", col=colorv[indeks],
+         add=as.logical(indeks-1))
+ } ## end for
```



```
> # Add title and legend
> title(main="Log-normal Distributions", line=0.5)
> legend("topright", inset=0.05, title="Sigmas",
+       paste("sigma", sigmavs, sep=""),
+       cex=0.8, lwd=2, lty=rep(1, NROW(sigmavs)),
+       col=colorv)
```

The Standard Deviation of Log-normal Prices

If percentage asset returns are *normally* distributed and follow *Brownian motion*, then asset prices follow *Geometric Brownian motion*, and they are *Log-normally* distributed at every point in time.

The standard deviation of *log-normal* prices is equal to the return volatility σ_r times the square root of time:
 $\sigma = \sigma_r \sqrt{t}$.

The *Log-normal* distribution has a strong positive skewness (third moment) equal to:

$$\varsigma = \mathbb{E}[(y - \mathbb{E}[y])^3] = (e^{\sigma^2} + 2)\sqrt{e^{\sigma^2} - 1}$$

For large standard deviation, the skewness increases exponentially with the standard deviation and with time: $\varsigma \propto e^{1.5\sigma^2} = e^{1.5\sigma_r^2 t}$

```
> # Return volatility of VTI etf
> sigmav <- sd(rutils::diffit(log(rutils::etfenv$VTI[, 4])))
> sigma2 <- sigmav^2
> nrows <- NROW(rutils::etfenv$VTI)
> # Standard deviation of log-normal prices
> sqrt(nrows)*sigmav
```



```
> # Skewness of log-normal prices
> calcskew <- function(t) {
+   expv <- exp(t*sigma2)
+   (expv + 2)*sqrt(expv - 1)
+ } ## end calcskew
> curve(expr=calcskew, xlim=c(1, nrows), lwd=3,
+ xlab="Number of days", ylab="Skewness", col="blue",
+ main="Skewness of Log-normal Prices
+ as a Function of Time")
```


The Mean and Median of *Log-normal* Prices

The mean of the *Log-normal* distribution:
 $\bar{y} = E[y] = \exp(\mu + \sigma^2/2)$ is greater than its median,
 which is equal to: $\tilde{y} = \exp(\mu)$.

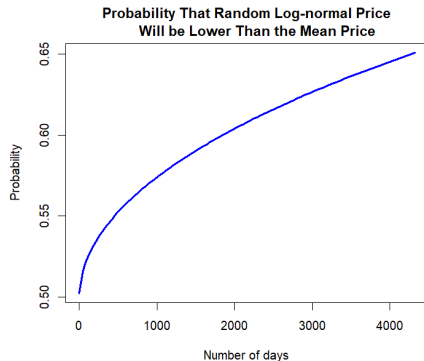
So if stock prices follow *Geometric Brownian motion*
 and are distributed *log-normally*, then a stock selected
 at random will have a high probability of having a lower
 price than the mean expected price.

The cumulative *Log-normal* probability distribution is
 equal to $F(x) = \Phi\left(\frac{\log y - \mu}{\sigma}\right)$, where $\Phi()$ is the
 cumulative standard normal distribution.

So the probability that the price of a randomly selected
 stock will be lower than the mean price is equal to
 $F(\bar{y}) = \Phi(\sigma/2)$.

Therefore an investor without skill, who selects stocks
 at random, has a high probability of underperforming
 the index.

Performing as well as the index requires *significant*
 investment skill, while outperforming the index requires
exceptional investment skill.



```
> # Probability that random log-normal price will be lower than the
> curve(expr=pnorm(sigmav*sqrt(x)/2),
+ xlim=c(1, nrow), lwd=3,
+ xlab="Number of days", ylab="Probability", col="blue",
+ main="Probability That Random Log-normal Price
+ Will be Lower Than the Mean Price")
```

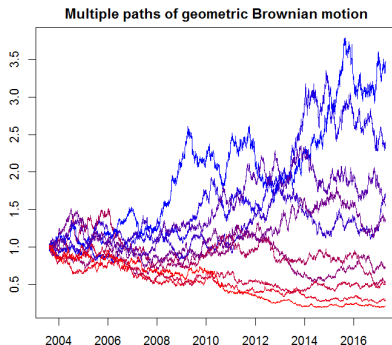
Paths of Geometric Brownian Motion

The standard deviation of *log-normal* prices σ is equal to the volatility of returns σ_r times the square root of time: $\sigma = \sigma_r \sqrt{t}$.

For large standard deviation, the skewness ς increases exponentially with the standard deviation and with

time: $\varsigma \propto e^{1.5\sigma^2} = e^{1.5t\sigma_r^2}$

```
> # Define the daily volatility and growth rate
> sigmav <- 0.01; drift <- 0.0; nrows <- 5000
> npaths <- 10
> # Simulate multiple paths of geometric Brownian motion
> pricev <- rnorm(npaths*nrows, sd=sigmav) + drift - sigmav^2/2
> pricev <- matrix(pricev, nc=npaths)
> pricev <- exp(matrixStats::colCumsums(pricev))
> # Create xts time series
> pricev <- xts(pricev, order.by=seq.Date(Sys.Date()-nrows+1, Sys.Date()),
+               order.by=seq.Date(Sys.Date()-nrows+1, Sys.Date()),
+               order.by=seq.Date(Sys.Date()-nrows+1, Sys.Date()))
> # Sort the columns according to largest terminal values
> pricev <- pricev[, order(pricev[nrows, ])]
> # Plot xts time series
> colorv <- colorRampPalette(c("red", "blue"))(NCOL(pricev))
> par(mar=c(3, 3, 2, 2), oma=c(0, 0, 0, 0))
> plot.zoo(pricev, main="Multiple paths of geometric Brownian motion",
+          xlab=NA, ylab=NA, plot.type="single", col=colorv)
```



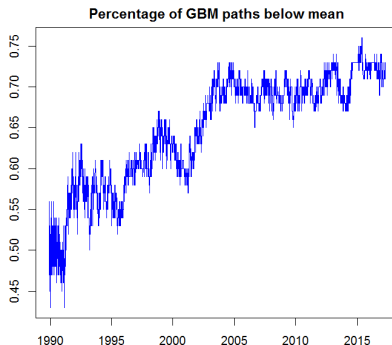
Distribution of Paths of Geometric Brownian Motion

Prices following *Geometric Brownian motion* have a large positive skewness, so that the expected value of prices is skewed by a few paths with very high prices, while the prices of the majority of paths are below their expected value.

For large standard deviation, the skewness ς increases exponentially with the standard deviation and with

$$\text{time: } \varsigma \propto e^{1.5\sigma^2} = e^{1.5t\sigma_r^2}$$

```
> # Define the daily volatility and growth rate
> sigmav <- 0.01; drift <- 0.0; nrows <- 10000
> npaths <- 100
> # Simulate multiple paths of geometric Brownian motion
> pricev <- rnorm(npaths*nrows, sd=sigmav) + drift - sigmav^2/2
> pricev <- matrix(pricev, nc=npaths)
> pricev <- exp(matrixStats::colCumsums(pricev))
> # Calculate fraction of paths below the expected value
> fractv <- rowSums(pricev < 1.0) / npaths
> # Create xts time series of percentage of paths below the expected value
> fractv <- xts(fractv, order.by=seq.Date(Sys.Date()-NROW(fractv)+1, Sys.Date(), by=1))
> # Plot xts time series of percentage of paths below the expected value
> par(mar=c(3, 3, 2, 2), oma=c(0, 0, 0, 0))
> plot.zoo(fractv, main="Percentage of GBM paths below mean",
+         xlab=NA, ylab=NA, col="blue")
```

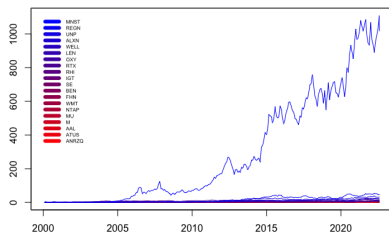


Time Evolution of Stock Prices

Stock prices evolve over time similar to *Geometric Brownian motion*, and they also exhibit a very skewed distribution of prices.

```
> # Load S&P500 stock prices
> load("/Users/jerzy/Develop/lecture_slides/data/sp500.RData")
> ls(sp500env)
> # Extract the closing prices
> pricev <- eapply(sp500env, quantmod::CL)
> # Flatten the prices into a single xts series
> pricev <- rutils::do_call(cbind, pricev)
> # Carry forward and backward non-NA prices
> pricev <- zoo::na.locf(pricev, na.rm=FALSE)
> pricev <- zoo::na.locf(pricev, fromLast=TRUE)
> sum(is.na(pricev))
> # Drop ".Close" from column names
> colnames(pricev)
> colnames(pricev) <- rutils::get_name(colnames(pricev))
> # Or
> # colnames(pricev) <- do.call(rbind,
> #   strsplit(colnames(pricev), split=".["))[, 1]
> # Select prices after the year 2000
> pricev <- pricev["2000/", ]
> # Scale the columns so that prices start at 1
> pricev <- lapply(pricev, function(x) x/as.numeric(x[1]))
> pricev <- rutils::do_call(cbind, pricev)
> # Sort the columns according to the final prices
> nrowv <- NROW(pricev)
> ordern <- order(pricev[nrowv, ])
> pricev <- pricev[, ordern]
> # Select 20 symbols
> symbolv <- colnames(pricev)
> symbolv <- symbolv[round(seq.int(from=1, to=NROW(symbolv), length.out=20))]
```

20 S&P500 Stock Prices (scaled)



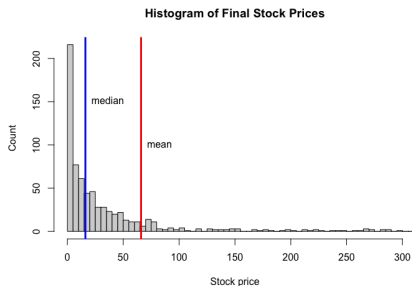
```
> # Plot xts time series of prices
> colorv <- colorRampPalette(c("red", "blue"))(NROW(symbolv))
> endd <- rutils::calc_endpoints(pricev, interval="weeks")
> plot.zoo(pricev[endd, symbolv], main="20 S&P500 Stock Prices (scaled)",
+   xlab=NA, ylab=NA, plot.type="single", col=colorv)
> legend(x="topleft", inset=0.02, cex=0.5, bty="n", y.intersp=0.5,
+   legend=rev(symbolv), col=rev(colorv), lwd=6, lty=1)
```

Distribution of Final Stock Prices

The distribution of the final stock prices is extremely skewed, with over 80% of the *S&P500* constituent stocks from 1990 now below the average price of that portfolio.

The *mean* of the final stock prices is much greater than the *median*.

```
> # Calculate the final stock prices
> pricef <- drop(zoo::coredata(pricev[nrows, ]))
> # Calculate the mean and median stock prices
> max(pricef); min(pricef)
> which.max(pricef)
> which.min(pricef)
> mean(pricef)
> median(pricef)
> # Calculate the percentage of stock prices below the mean
> sum(pricef < mean(pricef))/NROW(pricef)
```



```
> # Plot a histogram of final stock prices
> hist(pricef, breaks=1e3, xlim=c(0, 300),
+       xlab="Stock price", ylab="Count",
+       main="Histogram of Final Stock Prices")
> # Plot a histogram of final stock prices
> abline(v=median(pricef), lwd=3, col="blue")
> text(x=median(pricef), y=150, lab="median", pos=4)
> abline(v=mean(pricef), lwd=3, col="red")
> text(x=mean(pricef), y=100, lab="mean", pos=4)
```

Distribution of Stock Prices Over Time

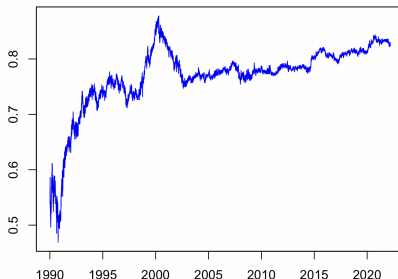
Usually, a small number of stocks in an index reach very high prices, while the prices of the majority of stocks remain below the index price (the average price of the index portfolio).

For example, the current prices of over 80% of the *S&P500* constituent stocks from 1990 are now below the average price of that portfolio.

Therefore an investor without skill, who selects stocks at random, has a high probability of underperforming the index, because they will most likely miss selecting the best performing stocks.

Performing as well as the index requires *significant* investment skill, while outperforming the index requires *exceptional* investment skill.

Percentage of S&P500 Stock Prices Below the Average Price



```
> # Calculate average of valid stock prices
> validp <- (pricev != 1) ## Valid stocks
> nstocks <- rowSums(validp)
> nstocks[1] <- NCOL(pricev)
> indeks <- rowSums(pricev*validp)/nstocks
> # Calculate fraction of stock prices below the average price
> fractv <- rowSums((pricev < indeks) & validp)/nstocks
> # Create xts time series of average stock prices
> indeks <- xts(indeks, order.by=zoo::index(pricev))
```

```
> dev.new(width=6, height=4, noRStudioGD=TRUE)
> # x11(width=6, height=4)
> # Plot xts time series of average stock prices
> plot.zoo(indeks, main="Average S&P500 Stock Prices (normalized fr
+   xlab=NA, ylab=NA, col="blue")
> # Create xts time series of percentage of stock prices below the a
> fractv <- xts(fractv, order.by=zoo::index(pricev))
> # Plot percentage of stock prices below the average price
> plot.zoo(fractv[-(1:2)],
+   main="Percentage of S&P500 Stock Prices
+   Below the Average Price",
+   xlab=NA, ylab=NA, col="blue")
```

Kernel Density of Asset Returns

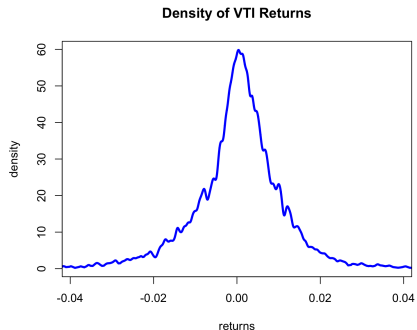
The kernel density is proportional to the number of data points close to a given point.

The kernel density is analogous to a histogram, but it provides more detailed information about the distribution of the data.

The smoothing kernel $K(x)$ is a symmetric function which decreases with the distance x .

The kernel density $d(r)$ at a point r is equal to the sum over the kernel function $K(x)$:

$$d(r) = \sum_{j=1}^n K(r - r_j)$$



```
> library(rutils) ## Load package rutils
> # Calculate VTI percentage returns
> retp <- rutils::etfenv$returns$VTI
> retp <- drop(coredata(na.omit(retp)))
> nrows <- NROW(retp)
> # Mean and standard deviation of returns
> c(mean(retp), sd(retp))
> # Calculate the smoothing bandwidth as the MAD of returns 10 poi
> retp <- sort(retp)
> bwidth <- 10*mad(rutils::diffit(retp, lagg=10))
> # Calculate the kernel density using a loop
> dens1 <- sapply(1:nrows, function(it) {
+   sum(dnorm(retp-retp[it], sd=bwidth))
+ })/nrows ## end sapply
```

```
> # Plot the kernel density
> madv <- mad(retp)
> plot(retp, dens1, xlim=c(-5*madv, 5*madv),
+       t="l", col="blue", lwd=3,
+       xlab="Returns", ylab="density",
+       main="Density of VTI Returns")
```

Kernel Density Using the Function density()

The function `density()` calculates a kernel estimate of the probability density for a sample of data.

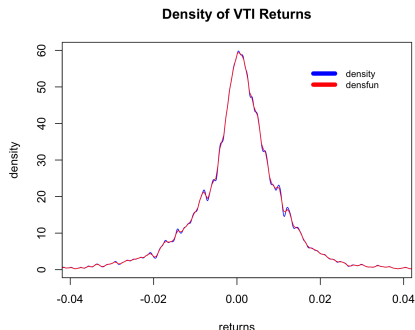
The parameter *smoothing bandwidth* is the standard deviation of the smoothing kernel $K(x)$.

The function `density()` returns a vector of densities at equally spaced points, not for the original data points.

The function `approx()` interpolates a vector of data into another vector.

The function `lines()` draws a line through specified points.

```
> # Calculate the kernel density using density()
> densv <- density(retp, bw=bandwidth)
> NROW(densv$y)
> plot(densv, xlim=c(-5*madv, 5*madv),
+      xlab="returns", ylab="density",
+      col="blue", lwd=3, main="Density of VTI Returns")
> # Interpolate the densv vector into returns
> densv <- approx(densv$x, densv$y, xout=retp)
> all.equal(densv$x, retp)
```



```
> # Plot the two density estimates
> plot(retp, dens1, xlim=c(-5*madv, 5*madv),
+      xlab="returns", ylab="density",
+      t="l", col="blue", lwd=1,
+      main="Density of VTI Returns")
> lines(retp, densv$y, col="red")
> # Add legend
> legend("topright", inset=0.05, cex=0.8, title=NULL,
+      leg=c("density", "densfun"), bty="n", y.intersp=0.4,
+      lwd=6, bg="white", col=c("blue", "red"))
```

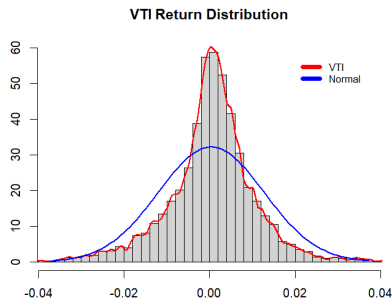

Distribution of Asset Returns

Asset returns are usually not normally distributed and they exhibit *leptokurtosis* (large kurtosis, or fat tails).

The function `hist()` calculates and plots a histogram, and returns its data *invisibly*.

The parameter `breaks` is the number of cells of the histogram.

The function `lines()` draws a line through specified points.



```
> # Plot histogram
> histp <- hist(retp, breaks=100, freq=FALSE,
+   xlim=c(-5*madv, 5*madv), xlab="", ylab="",
+   main="VTI Return Distribution")
> # Draw kernel density of histogram
> lines(densv, col="red", lwd=2)
> # Add density of normal distribution
> curve(expr=dnorm(x, mean=mean(retp), sd=sd(retp)),
+   add=TRUE, lwd=2, col="blue")
> # Add legend
> legend("topright", inset=0.05, cex=0.8, title=NULL,
+   leg=c("VTI", "Normal"), bty="n", y.intersp=0.4,
+   lwd=6, bg="white", col=c("red", "blue"))
```

The Quantile-Quantile Plot

A *Quantile-Quantile* (*Q-Q*) plot is a plot of points with the same *quantiles*, from two probability distributions.

If the two distributions are similar then all the points in the *Q-Q* plot lie along the diagonal.

The *VTI Q-Q* plot shows that the *VTI* return distribution has fat tails.

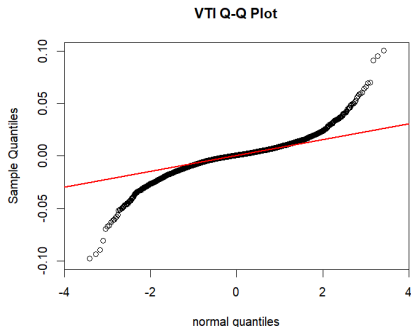
The *p*-value of the *Shapiro-Wilk* test is very close to zero, which shows that the *VTI* returns are very unlikely to be normal.

The function `shapiro.test()` performs the *Shapiro-Wilk* test of normality.

The function `qqnorm()` produces a normal *Q-Q* plot.

The function `qqline()` fits a line to the normal quantiles.

```
> # Create normal Q-Q plot
> qqnorm(retp, ylim=c(-0.1, 0.1), main="VTI Q-Q Plot",
+   xlab="Normal Quantiles")
> # Fit a line to the normal quantiles
> qqline(retp, col="red", lwd=2)
> # Perform Shapiro-Wilk test
> shapiro.test(retp)
```



Boxplots of Distributions of Values

Box-and-whisker plots (*boxplots*) are graphical representations of a distribution of values.

The bottom and top box edges (*hinges*) are equal to the first and third quartiles, and the *box* width is equal to the interquartile range (*IQR*).

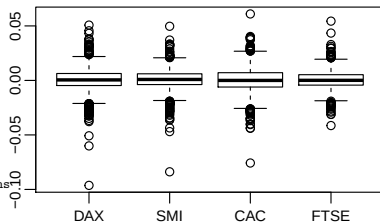
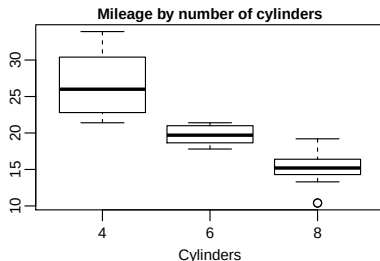
The nominal range is equal to 1.5 times the *IQR* above and below the box *hinges*.

The *whiskers* are dashed vertical lines representing values beyond the first and third quartiles, but within the nominal range.

The *whiskers* end at the last values within the nominal range, while the open circles represent outlier values beyond the nominal range.

The function `boxplot()` has two methods: one for formula objects (for categorical variables), and another for data frames.

```
> # Boxplot method for formula
> boxplot(formula=mpg ~ cyl, data=mtcars,
+   main="Mileage by number of cylinders",
+   xlab="Cylinders", ylab="Miles per gallon")
> # Boxplot method for data frame of EuStockMarkets percentage returns
> boxplot(x=diff(log(EuStockMarkets)))
```



Higher Moments of Asset Returns

The estimators of moments of a probability distribution are given by:

Sample mean: $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$

Sample variance: $\hat{\sigma}^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$

With their expected values equal to the population mean and standard deviation:

$\mathbb{E}[\bar{x}] = \mu$ and $\mathbb{E}[\hat{\sigma}] = \sigma$

The sample skewness (third moment):

$$\varsigma = \frac{n}{(n-1)(n-2)} \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{\hat{\sigma}} \right)^3$$

The sample kurtosis (fourth moment):

$$\kappa = \frac{n(n+1)}{(n-1)(n-2)(n-3)} \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{\hat{\sigma}} \right)^4$$

The normal distribution has skewness equal to 0 and kurtosis equal to 3.

Stock returns typically have negative skewness and kurtosis much greater than 3.

```
> # Calculate VTI percentage returns
> retp <- na.omit(rutils::etfenv$returns$VTI)
> # Number of observations
> nrow <- NROW(retp)
> # Mean of VTI returns
> retm <- mean(retp)
> # Standard deviation of VTI returns
> stdev <- sd(retp)
> # Skewness of VTI returns
> nrow/((nrow-1)*(nrow-2))*sum(((retp - retm)/stdev)^3)
> # Kurtosis of VTI returns
> nrow*(nrow+1)/((nrow-1)^3)*sum(((retp - retm)/stdev)^4)
> # Random normal returns
> retp <- rnorm(nrow, sd=stdev)
> # Mean and standard deviation of random normal returns
> retm <- mean(retp)
> stdev <- sd(retp)
> # Skewness of random normal returns
> nrow/((nrow-1)*(nrow-2))*sum(((retp - retm)/stdev)^3)
> # Kurtosis of random normal returns
> nrow*(nrow+1)/((nrow-1)^3)*sum(((retp - retm)/stdev)^4)
```

Functions for Calculating Skew and Kurtosis

R provides an easy way for users to write functions.

The function `calc_skew()` calculates the skew of returns, and `calc_kurt()` calculates the kurtosis.

Functions return the value of the last expression that is evaluated.

```
> # calc_skew() calculates skew of returns
> calc_skew <- function(retp) {
+   retp <- na.omit(retp)
+   sum(((retp - mean(retp))/sd(retp))^3)/NROW(retp)
+ } ## end calc_skew
> # calc_kurt() calculates kurtosis of returns
> calc_kurt <- function(retp) {
+   retp <- na.omit(retp)
+   sum(((retp - mean(retp))/sd(retp))^4)/NROW(retp)
+ } ## end calc_kurt
> # Calculate skew and kurtosis of VTI returns
> calc_skew(retp)
> calc_kurt(retp)
> # calc_mom() calculates the moments of returns
> calc_mom <- function(retp, moment=3) {
+   retp <- na.omit(retp)
+   sum(((retp - mean(retp))/sd(retp))^moment)/NROW(retp)
+ } ## end calc_mom
> # Calculate skew and kurtosis of VTI returns
> calc_mom(retp, moment=3)
> calc_mom(retp, moment=4)
```

Standard Errors of Estimators

Statistical estimators are functions of samples (which are random variables), and therefore are themselves *random variables*.

The *standard error* (SE) of an estimator is defined as its *standard deviation* (not to be confused with the *population standard deviation* of the underlying random variable).

For example, the *standard error* of the estimator of the mean is equal to:

$$\sigma_{\mu} = \frac{\sigma}{\sqrt{n}}$$

Where σ is the *population standard deviation* (which is usually unknown).

The *estimator* of this *standard error* is equal to:

$$SE_{\mu} = \frac{\hat{\sigma}}{\sqrt{n}}$$

where: $\hat{\sigma}^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$ is the sample standard deviation (the estimator of the population standard deviation).

```
> # Initialize the random number generator
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> # Sample from Standard Normal Distribution
> nrows <- 1000
> datav <- rnorm(nrows)
> # Sample mean
> mean(datav)
> # Sample standard deviation
> sd(datav)
> # Standard error of sample mean
> sd(datav)/sqrt(nrows)
```

Normal (Gaussian) Probability Distribution

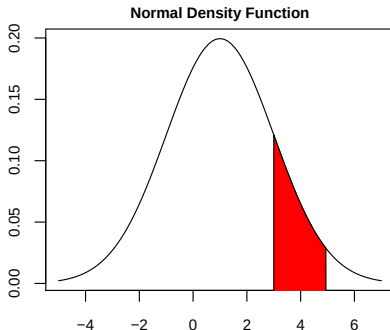
The *Normal (Gaussian)* probability density function is given by:

$$\phi(x, \mu, \sigma) = \frac{e^{-(x-\mu)^2/2\sigma^2}}{\sigma\sqrt{2\pi}}$$

The *Standard Normal* distribution $\phi(0, 1)$ is a special case of the *Normal* $\phi(\mu, \sigma)$ with $\mu = 0$ and $\sigma = 1$.

The function `dnorm()` calculates the *Normal* probability density.

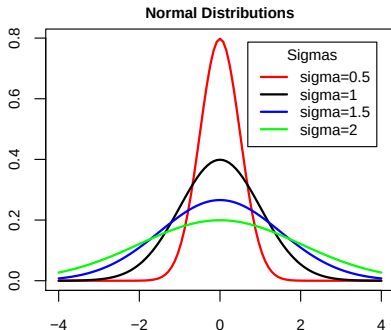
```
> xvar <- seq(-5, 7, length=100)
> yvar <- dnorm(xvar, mean=1.0, sd=2.0)
> plot(xvar, yvar, type="l", lty="solid", xlab="", ylab="")
> title(main="Normal Density Function", line=0.5)
> startp <- 3; endd <- 5 ## Set lower and upper bounds
> # Set polygon base
> subv <- ((xvar >= startp) & (xvar <= endd))
> polygon(c(startp, xvar[subv], endd), ## Draw polygon
+ c(-1, yvar[subv], -1), col="red")
```



Normal (Gaussian) Probability Distributions

Plots of several *Normal* distributions with different values of σ , using the function `curve()` for plotting functions given by their name.

```
> sigmavs <- c(0.5, 1, 1.5, 2) ## Sigma values
> # Create plot colors
> colorv <- c("red", "black", "blue", "green")
> # Create legend labels
> labelv <- paste("sigma", sigmavs, sep="")
> for (it in 1:4) { ## Plot four curves
+   curve(expr=dnorm(x, sd=sigmavs[it]),
+         xlim=c(-4, 4), xlab="", ylab="", lwd=2,
+         col=colorv[it], add=as.logical(it-1))
+ } ## end for
> # Add title
> title(main="Normal Distributions", line=0.5)
> # Add legend
> legend("topright", inset=0.05, title="Sigmas", y.intersp=0.4,
+       labelv, cex=0.8, lwd=2, lty=1, bty="n", col=colorv)
```



Student's t -distribution

Let $z_1, \dots, z_\nu$ be independent standard normal random variables, with sample mean: $\bar{z} = \frac{1}{\nu} \sum_{i=1}^{\nu} z_i$ ($\mathbb{E}[\bar{z}] = \mu$) and sample variance:

$$\hat{\sigma}^2 = \frac{1}{\nu-1} \sum_{i=1}^{\nu} (z_i - \bar{z})^2$$

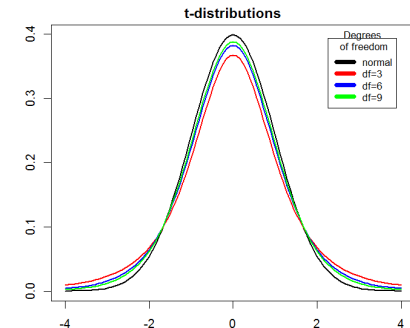
Then the random variable (t -ratio):

$$t = \frac{\bar{z} - \mu}{\hat{\sigma} / \sqrt{\nu}}$$

Follows the t -distribution with ν degrees of freedom, with the probability density function:

$$f(t) = \frac{\Gamma((\nu + 1)/2)}{\sqrt{\pi\nu} \Gamma(\nu/2)} (1 + t^2/\nu)^{-(\nu+1)/2}$$

```
> degf <- c(3, 6, 9) ## Df values
> colorv <- c("black", "red", "blue", "green")
> labelv <- c("normal", paste("df", degf, sep=" "))
> # Plot a Normal probability distribution
> curve(expr=dnorm, xlim=c(-4, 4), xlab="", ylab="", lwd=2)
> for (it in 1:3) { ## Plot three t-distributions
+   curve(expr=dt(x, df=degf[it]), xlab="", ylab="",
+   lwd=2, col=colorv[it+1], add=TRUE)
+ } ## end for
```



```
> # Add title
> title(main="t-distributions", line=0.5)
> # Add legend
> legend("topright", inset=0.05, bty="n", y.intersp=0.4,
+       title="Degrees\n of freedom", labelv,
+       cex=0.8, lwd=6, lty=1, col=colorv)
```

Mixture Models of Returns

Mixture models are produced by randomly sampling data from different distributions.

The mixture of two normal distributions with different variances produces a distribution with *leptokurtosis* (large kurtosis, or fat tails).

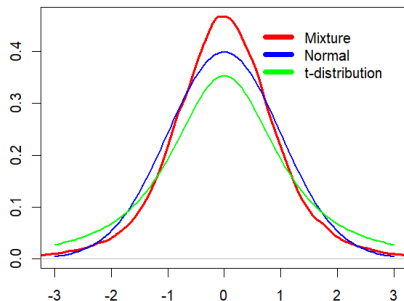
Student's *t-distribution* has fat tails because the sample variance in the denominator of the *t-ratio* is variable.

The time-dependent volatility of asset returns is referred to as *heteroskedasticity*.

Random processes with *heteroskedasticity* can be considered a type of mixture model.

The *heteroskedasticity* produces *leptokurtosis* (large kurtosis, or fat tails).

Mixture of Normal Returns



```
> # Mixture of two normal distributions with sd=1 and sd=2
> nrows <- 1e5
> retp <- c(rnorm(nrows/2), 2*rnorm(nrows/2))
> retp <- (retp-mean(retp))/sd(retp)
> # Kurtosis of normal
> calc_kurt(rnorm(nrows))
> # Kurtosis of mixture
> calc_kurt(retp)
> # Or
> nrows*sum(retp^4)/(nrows-1)^2
```

```
> # Plot the distributions
> plot(density(retp), xlab="", ylab="",
+      main="Mixture of Normal Returns",
+      xlim=c(-3, 3), type="l", lwd=3, col="red")
> curve(expr=dnorm, lwd=2, col="blue", add=TRUE)
> curve(expr=dt(x, df=3), lwd=2, col="green", add=TRUE)
> # Add legend
> legend("topright", inset=0.05, lty=1, lwd=6, bty="n",
+      legend=c("Mixture", "Normal", "t-distribution"), y.intersp=0.4,
+      col=c("red", "blue", "green"))
```

Non-standard Student's *t*-distribution

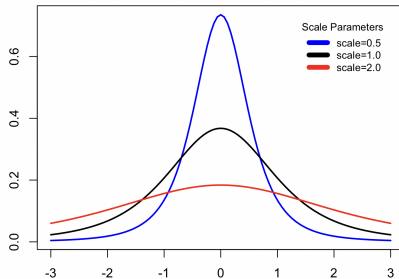
The non-standard Student's *t*-distribution has the probability density function:

$$f(t) = \frac{\Gamma((\nu + 1)/2)}{\sqrt{\pi\nu} \sigma \Gamma(\nu/2)} \left(1 + \left(\frac{t - \mu}{\sigma}\right)^2 / \nu\right)^{-(\nu+1)/2}$$

It has a non-zero mean equal to the location parameter μ , and a standard deviation proportional to the scale parameter σ .

```
> dev.new(width=6, height=5, noRStudioGD=TRUE)
> # x11(width=6, height=5)
> # Define density of non-standard t-distribution
> tdistr <- function(x, dfree, locv=0, scalev=1) {
+   dt((x-locv)/scalev, df=dfree)/scalev
+ } ## end tdistr
> # Or
> tdistr <- function(x, dfree, locv=0, scalev=1) {
+   gamma((dfree+1)/2)/(sqrt(pi*dfree)*gamma(dfree/2)*scalev)*
+   (1+((x-locv)/scalev)^2/dfree)^(-(dfree+1)/2)
+ } ## end tdistr
> # Calculate vector of scale values
> scalev <- c(0.5, 1.0, 2.0)
> colorv <- c("blue", "black", "red")
> labelv <- paste("scale", format(scalev, digits=2), sep="")
> # Plot three t-distributions
> for (it in 1:3) {
+   curve(expr=tdistr(x, dfree=3, scalev=scalev[it]), xlim=c(-3, 3),
+   xlab="", ylab="", lwd=2, col=colorv[it], add=(it>1))
+ } ## end for
```

t-distributions with Different Scale Parameters



```
> # Add title
> title(main="t-distributions with Different Scale Parameters", line=1)
> # Add legend
> legend("topright", inset=0.05, bty="n", title="Scale Parameters",
+       cex=0.8, lwd=6, lty=1, col=colorv, y.intersp=0.4)
```

The *Shapiro-Wilk* Test of Normality

The *Shapiro-Wilk* test is designed to test the *null hypothesis* that a sample: $\{x_1, \dots, x_n\}$ is from a normally distributed population.

The test statistic is equal to:

$$W = \frac{(\sum_{i=1}^n a_i x_{(i)})^2}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

Where the: $\{a_1, \dots, a_n\}$ are proportional to the *order statistics* of random variables from the normal distribution.

$x_{(k)}$ is the *k*-th *order statistic*, and is equal to the *k*-th smallest value in the sample: $\{x_1, \dots, x_n\}$.

The *Shapiro-Wilk* statistic follows its own distribution, and is less than or equal to 1.

The *Shapiro-Wilk* statistic is close to 1 for samples from normal distributions.

The *p*-value for *VTI* returns is extremely small, and we conclude that the *null hypothesis* is FALSE, and the *VTI* returns are not from a normally distributed population.

The *Shapiro-Wilk* test is not reliable for large sample sizes, so it's limited to less than 5000 sample size.

```
> # Calculate VTI percentage returns
> library(rutils)
> retp <- as.numeric(na.omit(rutils::etfenv$returns$VTI))[1:499]
> # Reduce number of output digits
> ndigits <- options(digits=5)
> # Shapiro-Wilk test for normal distribution
> nrows <- NROW(retp)
> shapiro.test(rnorm(nrows))
```

Shapiro-Wilk normality test

```
data:  rnorm(nrows)
W = 0.998, p-value = 0.76
> # Shapiro-Wilk test for VTI returns
> shapiro.test(retp)
```

Shapiro-Wilk normality test

```
data:  retp
W = 0.991, p-value = 0.0029
> # Shapiro-Wilk test for uniform distribution
> shapiro.test(runif(nrows))
```

Shapiro-Wilk normality test

```
data:  runif(nrows)
W = 0.948, p-value = 3.1e-12
> # Restore output digits
> options(digits=ndigits$digits)
```

The Jarque-Bera Test of Normality

The *Jarque-Bera* test is designed to test the *null hypothesis* that a sample: $\{x_1, \dots, x_n\}$ is from a normally distributed population.

The test statistic is equal to:

$$JB = \frac{n}{6} \left(\varsigma^2 + \frac{1}{4} (\kappa - 3)^2 \right)$$

Where the *skewness* and *kurtosis* are defined as:

$$\varsigma = \frac{1}{n} \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{\hat{\sigma}} \right)^3 \quad \kappa = \frac{1}{n} \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{\hat{\sigma}} \right)^4$$

The *Jarque-Bera* statistic asymptotically follows the *chi-squared* distribution with 2 degrees of freedom.

The *Jarque-Bera* statistic is small for samples from normal distributions.

The *p*-value for *VTI* returns is extremely small, and we conclude that the *null hypothesis* is FALSE, and the *VTI* returns are not from a normally distributed population.

```
> library(tseries) ## Load package tseries
> # Jarque-Bera test for normal distribution
> jarque.bera.test(rnorm(nrows))
```

Jarque Bera Test

```
data:  rnorm(nrows)
X-squared = 5, df = 2, p-value = 0.1
> # Jarque-Bera test for VTI returns
> jarque.bera.test(retp)
```

Jarque Bera Test

```
data:  retp
X-squared = 22, df = 2, p-value = 2e-05
> # Jarque-Bera test for uniform distribution
> jarque.bera.test(runif(NROW(retp)))
```

Jarque Bera Test

```
data:  runif(NROW(retp))
X-squared = 26, df = 2, p-value = 3e-06
```

The Kolmogorov-Smirnov Test for Probability Distributions

The *Kolmogorov-Smirnov* test *null hypothesis* is that two samples: $\{x_1, \dots, x_n\}$ and $\{y_1, \dots, y_n\}$ were obtained from the same probability distribution.

The *Kolmogorov-Smirnov* statistic depends on the maximum difference between two empirical cumulative distribution functions (cumulative frequencies):

$$D = \sup_i |P(x_i) - P(y_i)|$$

The function `ks.test()` performs the *Kolmogorov-Smirnov* test and returns the statistic and its *p*-value *invisibly*.

The second argument to `ks.test()` can be either a numeric vector of data values, or a name of a cumulative distribution function.

The *Kolmogorov-Smirnov* test can be used as a *goodness of fit* test, to test if a set of observations fits a probability distribution.

```
> # KS test for normal distribution
> kstest <- ks.test(rnorm(100), pnorm)
> kstest$p.value
> # KS test for uniform distribution
> ks.test(runif(100), pnorm)
> # KS test for two shifted normal distributions
> ks.test(rnorm(100), rnorm(100, mean=0.1))
> ks.test(rnorm(100), rnorm(100, mean=1.0))
> # KS test for two different normal distributions
> ks.test(rnorm(100), rnorm(100, sd=2.0))
> # KS test for VTI returns vs normal distribution
> retp <- as.numeric(na.omit(rutils::etfenv$returns$VTI))
> retp <- (retp - mean(retp))/sd(retp)
> ks.test(retp, pnorm)
```

Chi-squared Distribution

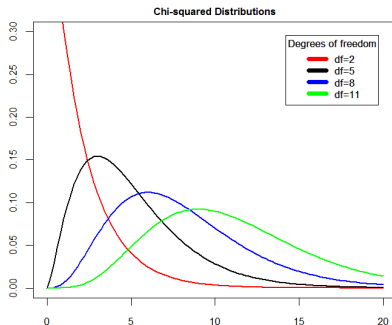
Let $z_1, \dots, z_k$ be independent standard *Normal* random variables.

Then the random variable $X = \sum_{i=1}^k z_i^2$ is distributed according to the *Chi-squared* distribution with k degrees of freedom: $X \sim \chi_k^2$, and its probability density function is given by:

$$f(x) = \frac{x^{k/2-1} e^{-x/2}}{2^{k/2} \Gamma(k/2)}$$

The *Chi-squared* distribution with k degrees of freedom has mean equal to k and variance equal to $2k$.

```
> # Degrees of freedom
> degf <- c(2, 5, 8, 11)
> # Plot four curves in loop
> colorv <- c("red", "black", "blue", "green")
> for (it in 1:4) {
+   curve(expr=dchisq(x, df=degf[it]),
+         xlim=c(0, 20), ylim=c(0, 0.3),
+         xlab="", ylab="", col=colorv[it],
+         lwd=2, add=as.logical(it-1))
+ } ## end for
```



```
> # Add title
> title(main="Chi-squared Distributions", line=0.5)
> # Add legend
> labelv <- paste("df", degf, sep="=")
> legend("topright", inset=0.05, bty="n", y.intersp=0.4,
+       title="Degrees of freedom", labelv,
+       cex=0.8, lwd=6, lty=1, col=colorv)
```

The *Chi-squared* Test for the Goodness of Fit

Goodness of Fit tests are designed to test if a set of observations fits an assumed theoretical probability distribution.

The *Chi-squared* test tests if a frequency of counts fits the specified distribution.

The *Chi-squared* statistic is the sum of squared differences between the observed frequencies o_i and the theoretical frequencies p_i :

$$\chi^2 = N \sum_{i=1}^n \frac{(o_i - p_i)^2}{p_i}$$

Where N is the total number of observations.

The *null hypothesis* is that the observed frequencies are consistent with the theoretical distribution.

The function `chisq.test()` performs the *Chi-squared* test and returns the statistic and its *p*-value *invisibly*.

The parameter `breaks` in the function `hist()` should be chosen large enough to capture the shape of the frequency distribution.

```
> # Observed frequencies from random normal data
> histp <- hist(rnorm(1e3, mean=0), breaks=100, plot=FALSE)
> countsn <- histp$counts
> # Theoretical frequencies
> countst <- rutils::diffit(pnorm(histp$breaks))
> # Perform Chi-squared test for normal data
> chisq.test(x=countsn, p=countst, rescale.p=TRUE, simulate.p.value=TRUE)
> # Return p-value
> chisqtest <- chisq.test(x=countsn, p=countst, rescale.p=TRUE, simulate.p.value=TRUE)
> chisqtest$p.value
> # Observed frequencies from shifted normal data
> histp <- hist(rnorm(1e3, mean=2), breaks=100, plot=FALSE)
> countsn <- histp$counts/sum(histp$counts)
> # Theoretical frequencies
> countst <- rutils::diffit(pnorm(histp$breaks))
> # Perform Chi-squared test for shifted normal data
> chisq.test(x=countsn, p=countst, rescale.p=TRUE, simulate.p.value=TRUE)
> # Calculate histogram of VTI returns
> histp <- hist(retp, breaks=100, plot=FALSE)
> countsn <- histp$counts
> # Calculate cumulative probabilities and then difference them
> countst <- pt((histp$breaks-locv)/scalev, df=2)
> countst <- rutils::diffit(countst)
> # Perform Chi-squared test for VTI returns
> chisq.test(x=countsn, p=countst, rescale.p=TRUE, simulate.p.value=TRUE)
```


The Likelihood Function of Student's *t*-distribution

The non-standard Student's *t*-distribution is:

$$f(t) = \frac{\Gamma((\nu + 1)/2)}{\sqrt{\pi\nu} \sigma \Gamma(\nu/2)} \left(1 + \left(\frac{t - \mu}{\sigma}\right)^2 / \nu\right)^{-(\nu+1)/2}$$

It has non-zero mean equal to the location parameter μ , and a standard deviation proportional to the scale parameter σ .

The negative logarithm of the probability density is equal to:

$$-\log(f(t|\mu, \sigma)) = -\log\left(\frac{\Gamma((\nu + 1)/2)}{\sqrt{\pi\nu} \Gamma(\nu/2)}\right) + \log(\sigma) + \frac{\nu + 1}{2} \log\left(1 + \left(\frac{t - \mu}{\sigma}\right)^2 / \nu\right)$$

The *likelihood* function $\mathcal{L}(\mu, \sigma|\mathbf{t})$ is the product of the individual probability density functions $f(t_i|\mu, \sigma)$:

$$\mathcal{L}(\mu, \sigma|\mathbf{t}) = \prod_{i=1}^n f(t_i|\mu, \sigma)$$

The *likelihood* $\mathcal{L}(\mu, \sigma|\mathbf{t})$ is a function of the model parameters μ and σ , given the vector of the observed values $\mathbf{t}$, under the model's probability distribution $f(t|\mu, \sigma)$:

```
> # Objective function from function dt()
> likefun <- function(par, dfree, datav) {
+   -sum(log(dt(x=(datav-par[1])/par[2], df=dfree)/par[2]))
+ } ## end likefun
> # Demonstrate equivalence with log(dt())
> likefun(c(1, 0.5), 2, 2:5)
> -sum(log(dt(x=2:5-1)/0.5, df=2)/0.5))
> # Objective function is negative log-likelihood
> likefun <- function(par, dfree, datav) {
+   sum(-log(gamma((dfree+1)/2)/(sqrt(pi*dfree)*gamma(dfree/2))) +
+     log(par[2]) + (dfree+1)*2*log(1+((datav-par[1])/par[2])^2/dfree))
+ } ## end likefun
```

The *likelihood* function measures how *likely* are the model parameters μ, σ , given the observed values $\mathbf{t}$.

The *maximum-likelihood* estimate (MLE) of the parameters $\theta = (\mu, \sigma)$ are those that maximize the *likelihood* function:

$$\theta_{MLE} = \arg \max_{\theta} \mathcal{L}(\theta|\mathbf{t})$$

In practice the logarithm of the *likelihood* $\log(\mathcal{L})$ is maximized, instead of the *likelihood* itself.

Fitting Asset Returns into Student's t -distribution

The function `fitdistr()` from package *MASS* fits a univariate distribution to a sample of data, by performing *maximum likelihood* optimization.

The function `fitdistr()` performs a *maximum likelihood* optimization to find the non-standardized Student's t -distribution location and scale parameters.

```
> # Calculate VTI percentage returns
> retp <- as.numeric(na.omit(rutils::etfenv$returns$VTI))
> # Fit VTI returns using MASS::fitdistr()
> fitobj <- MASS::fitdistr(retp, densfun="t", df=3)
> summary(fitobj)
> # Fitted parameters
> fitobj$estimate
> locv <- fitobj$estimate[1]
> scalev <- fitobj$estimate[2]
> locv; scalev
> # Standard errors of parameters
> fitobj$sed
> # Log-likelihood value
> fitobj$value
> # Fit distribution using optim()
> initp <- c(mean=0, scale=0.01) ## Initial parameters
> fitobj <- optim(par=initp,
+   fn=likefun, ## Log-likelihood function
+   datav=retp,
+   dfree=3, ## Degrees of freedom
+   method="L-BFGS-B", ## Quasi-Newton method
+   upper=c(1, 0.1), ## Upper constraint
+   lower=c(-1, 1e-7)) ## Lower constraint
> # Optimal parameters
> locv <- fitobj$par["mean"]
> scalev <- fitobj$par["scale"]
> locv; scalev
```

The Student's t -distribution Fitted to Asset Returns

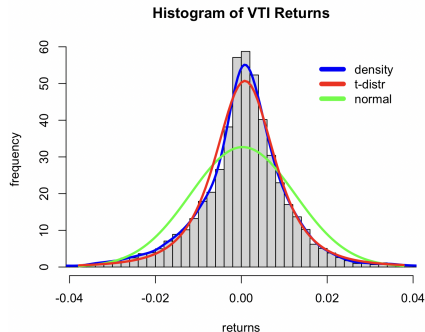
Asset returns typically exhibit *negative skewness* and *large kurtosis* (leptokurtosis), or fat tails.

Stock returns fit the non-standard t -distribution with 3 degrees of freedom quite well.

The function `hist()` calculates and plots a histogram, and returns its data *invisibly*.

The parameter `breaks` is the number of cells of the histogram.

```
> dev.new(width=6, height=5, noRStudioGD=TRUE)
> # x11(width=6, height=5)
> # Plot histogram of VTI returns
> madv <- mad(retp)
> histp <- hist(retp, col="lightgrey",
+   xlab="returns", breaks=100, xlim=c(-5*madv, 5*madv),
+   ylab="frequency", freq=FALSE, main="Histogram of VTI Returns")
> lines(density(retp, adjust=1.5), lwd=3, col="blue")
> # Plot the Normal probability distribution
> curve(expr=dnorm(x, mean=mean(retp),
+   sd=sd(retp)), add=TRUE, lwd=3, col="green")
> # Define non-standard t-distribution
> tdistr <- function(x, dfree, locv=0, scalev=1) {
+   dt((x-locv)/scalev, df=dfree)/scalev
+ } ## end tdistr
> # Plot t-distribution function
> curve(expr=tdistr(x, dfree=3, locv=locv, scalev=scalev), col="red", lwd=3, add=TRUE)
> # Add legend
> legend("topright", inset=0.05, bty="n", y.intersp=0.4,
+   leg=c("density", "t-distr", "normal"),
+   lwd=6, lty=1, col=c("blue", "red", "green"))
```



Goodness of Fit of Student's t -distribution Fitted to Asset Returns

The Q-Q plot illustrates the relative distributions of two samples of data.

The Q-Q plot shows that stock returns fit the non-standard t -distribution with 3 degrees of freedom quite well.

The function `qqplot()` produces a Q-Q plot for two samples of data.

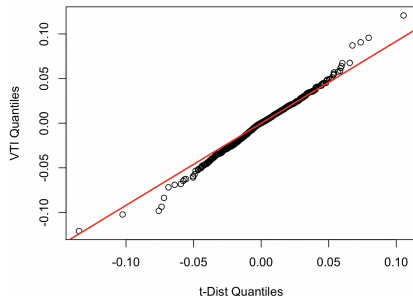
The function `ks.test()` performs the Kolmogorov-Smirnov test for the similarity of two distributions.

The null hypothesis of the Kolmogorov-Smirnov test is that the two samples were obtained from the same probability distribution.

The Kolmogorov-Smirnov test rejects the null hypothesis that stock returns follow closely the non-standard t -distribution with 3 degrees of freedom.

```
> # Calculate sample from non-standard t-distribution with df=3
> datat <- locv + scalev*rt(NROW(retp), df=3)
> # KS test for VTI returns vs t-distribution data
> ks.test(retp, datat)
```

Q-Q plot of VTI Returns vs Student's t -distribution



```
> # Q-Q plot of VTI Returns vs non-standard t-distribution
> qqplot(datat, retp, xlab="t-Dist Quantiles", ylab="VTI Quantiles",
+       main="Q-Q plot of VTI Returns vs Student's t-distribution")
> # Calculate quantiles of the distributions
> probs <- c(0.25, 0.75)
> qrets <- quantile(retp, probs)
> qtdata <- quantile(datat, probs)
> # Calculate slope and plot line connecting quantiles
> slope <- diff(qrets)/diff(qtdata)
> intercept <- qrets[1]-slope*qtdata[1]
> abline(intercept, slope, lwd=2, col="red")
```

Leptokurtosis Fat Tails of Asset Returns

The probability under the *normal* distribution decreases exponentially for large values of x :

$$\phi(x) \propto e^{-x^2/2\sigma^2} \quad (\text{as } |x| \rightarrow \infty)$$

This is because a normal variable can be thought of as the sum of a large number of independent binomial variables of equal size.

So large values are produced only when all the contributing binomial variables are of the same sign, which is very improbable, so it produces extremely low tail probabilities (thin tails),

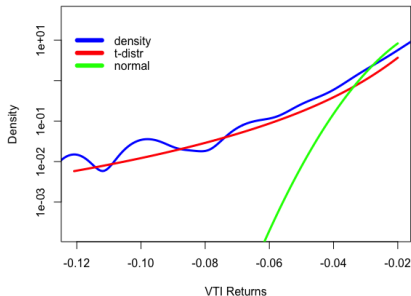
But in reality, the probability of large negative asset returns decreases much slower, as the negative power of the returns (fat tails).

The probability under Student's *t-distribution* decreases as a power for large values of x :

$$f(x) \propto |x|^{-(\nu+1)} \quad (\text{as } |x| \rightarrow \infty)$$

This is because a *t-variable* can be thought of as the sum of normal variables with different volatilities (different sizes).

Fat Left Tail of VTI Returns (density in log scale)



```
> # Plot log density of VTI returns
> plot(density(retp, adjust=4), xlab="VTI Returns", ylab="Density",
+      main="Fat Left Tail of VTI Returns (density in log scale)",
+      type="l", lwd=3, col="blue", xlim=c(min(retp), -0.02), log="y")
> # Plot t-distribution function
> curve(expr=dt((x-locv)/scalev, df=3)/scalev, lwd=3, col="red", add=TRUE)
> # Plot the Normal probability distribution
> curve(expr=dnorm(x, mean=mean(retp), sd=sd(retp)), lwd=3, col="green", add=TRUE)
> # Add legend
> legend("topleft", inset=0.01, bty="n", y.intersp=c(0.25, 0.25, 0.25),
+       legend=c("density", "t-distr", "normal"), y.intersp=0.4,
+       lwd=6, lty=1, col=c("blue", "red", "green"))
```

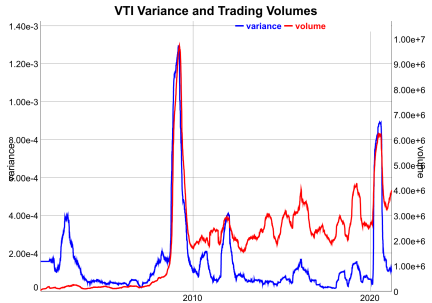
Trading Volumes

The average trading volumes have increased significantly since the 2008 crisis, mostly because of high frequency trading (HFT).

Higher levels of volatility coincide with higher *trading volumes*.

The time-dependent volatility of asset returns (*heteroskedasticity*) produces their fat tails (*leptokurtosis*).

```
> # Calculate VTI returns and trading volumes
> ohlc <- rutils::etfenv$VTI
> closep <- drop(coredata(quantmod::Cl(ohlc)))
> retp <- rutils::diffit(log(closep))
> volumv <- coredata(quantmod::Vo(ohlc))
> # Calculate trailing variance
> lookb <- 121
> varv <- HighFreq::roll_var_ohlc(log(ohlc), method="close", lookb=lookb, scale=FALSE)
> varv[1:lookb, ] <- varv[lookb+1, ]
> # Calculate trailing average volume
> volumr <- HighFreq::roll_sum(volumv, lookb=lookb)/lookb
> # dygraph plot of VTI variance and trading volumes
> datav <- xts::xts(cbind(varv, volumr), zoo::index(ohlc))
> colv <- c("variance", "volume")
> colnames(datav) <- colv
> dygraphs::dygraph(datav, main="VTI Variance and Trading Volumes") %>%
+   dyAxis("y", label=colv[1], independentTicks=TRUE) %>%
+   dyAxis("y2", label=colv[2], independentTicks=TRUE) %>%
+   dySeries(name=colv[1], strokeWidth=2, axis="y", col="blue") %>%
+   dySeries(name=colv[2], strokeWidth=2, axis="y2", col="red") %>%
+   dyLegend(show="always", width=500)
```



Asset Returns in Trading Time

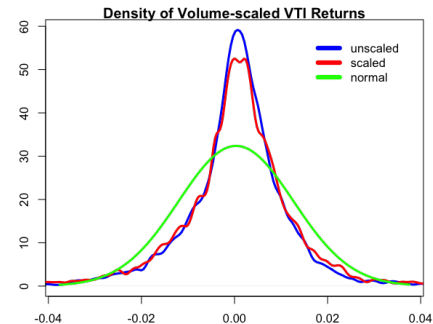
The time-dependent volatility of asset returns (*heteroskedasticity*) produces their fat tails (*leptokurtosis*).

If asset returns were measured at fixed intervals of *trading volumes* (*trading time* instead of clock time), then the volatility would be lower and less time-dependent.

The asset returns can be adjusted to *trading time* by dividing them by the *square root of the trading volumes*, to obtain scaled returns over equal trading volumes.

The scaled returns have a more positive *skewness* and a smaller *kurtosis* than unscaled returns.

```
> # Scale the returns using volume clock to trading time
> retsc <- ifelse(volumv > 0, sqrt(volumv)*retp/sqrt(volumv), 0)
> retsc <- sd(retp)*retsc/sd(retsc)
> # retsc <- ifelse(volumv > 1e4, retp/volumv, 0)
> # Calculate moments of scaled returns
> nrow <- NROW(retp)
> sapply(list(retp=retp, retsc=retsc),
+   function(rets) {sapply(c(skew=3, kurt=4),
+     function(x) sum((rets/sd(rets))^x)/nrow)
+ }) ## end sapply
```



```
> # x11(width=6, height=5)
> dev.new(width=6, height=5, noRStudioGD=TRUE)
> par(mar=c(3, 3, 2, 1), oma=c(1, 1, 1, 1))
> # Plot densities of SPY returns
> madv <- mad(retp)
> # bwidh <- mad(rutils::diffit(retp))
> plot(density(retp, bw=madv/10), xlim=c(-5*madv, 5*madv),
+   lwd=3, mgp=c(2, 1, 0), col="blue",
+   xlab="returns (standardized)", ylab="frequency",
+   main="Density of Volume-scaled VTI Returns")
> lines(density(retsc, bw=madv/10), lwd=3, col="red")
> curve(expr=dnorm(x, mean=mean(retp), sd=sd(retp)),
+   add=TRUE, lwd=3, col="green")
> # Add legend
> legend("topright", inset=0.05, bty="n", y.intersp=0.4
```

Homework Assignment

Required

- Study all the lecture slides in *FRE6871_Lecture_4.pdf*, and run all the code in *FRE6871_Lecture_4.R*